Amlogic

SDK 使用指南(Android S)

Revision: 01

Release Date: 2022-08-30

Copyright

Trademarks

 , and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time.

This product is not intended for use in medical, life saving, or life sustaining applications.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision History

Issue 01 (2022-08-30)

This is the first official release.

Contents

1. 文档所覆盖的 code base

此开发文档只针对如下 code base，以及相应的 SOC

Codebase	描述	SOC
Android S SDK	Android12.0\Linux5.4	S905X4/S905Y4/S805X2..

参考文档：Android_S_release_notes_vxxxxxxxx.pdf

2. SDK 环境和使用

2.1 编译开发环境要求

- 建议 Linux 发行版本是 Ubuntu 16.04 server 64bit 或者更新的版本
- Android 相关的 JDK 相关的工具下载和编译，参考以下链接

https://source.android.com/source/initializing.htmlhttps://source.android.com/source/initializing.html

- 联系对应的技术支持窗口,获取如下文档:

文档	Link
Android P&Q Environment Building Guide	https://doc.amlogic.com/file/detail?type=1&id=4105
Android R&S Environment Building Guide	https://doc.amlogic.com/file/detail?type=1&id=17200

2.2 SDK 介绍

代码目录结构

Android 根目录：

```
├── art                 //ART 运行环境
├── bionic              //系统 C 库
├── bootable            //android recovery
├── bootloader          //uboot
├── build               //Android Makefile
├── common              //Linux kernel
├── compatibility
├── cts                 //CTS 测试源码
├── dalvik              //dalvik 虚拟机
├── developers          //应用开发者相关目录
├── development         //应用开发者相关目录
├── device              //平台相关的文件和配置
├── extrnal             //第三方开源库
├── frameworks          //Android 系统框架
├── hardware            //硬件相关的 HAL 代码
├── kernel              //Android kernel 相关
├── libcore             //系统核心库
├── libnativehelper     //动态库，实现 JNI 库的基础
├── packages            //Android 原生 app 代码
├── pdk                 //本地开发套件
├── platform_testing    //测试程序
├── prebuilts           //x86 和 arm 架构的预编译资源
├── sdk                 //sdk 和模拟器
├── system              //Linux 底层系统库、应用和组件
├── test                //vts sts 测试相关
├── toolchain           //工具连
├── tools               //工具程序
```

```
├── vendor                  //vendor 厂商代码
└── out                         //编译输出目录
```

Uboot 目录结构

bootloader/uboot-repo/

```
├── bl2
├── bl30
├── bl31
├── bl31_1.3
├── bl32
├── bl32_3.8
├── bl33
└── v2019
├──  board/amlogic/configs    //各种板宏、环境变量配置
├──  board/amlogic/s4_ap222  //S905Y4 公版 AP222 板子相关
├──  cmd                          //uboot 下各种命令代码
├──  drivers                          //uboot  下驱动目录
├── bl40
├── fip          //编译工具
```

BL2: Boot Loader 2 which is the first external software loaded and executed by the SoC

BL30: SCP firmware

BL31: ARM Trusted Firmware

BL32: TEE or Secure OS

BL33:Loader responsible to load kernel into DDR to execute. In Amlogic reference software, BL33 will be U-boot

BL4: M4 boot ROM

BL40: M4 firmware

Linux kernel 目录结构：

common/

```
├── arch
├── arm64/boot/dts/amlogic          //板子 DTS 文件目录
├── block
├── certs
├── crypto
├── Documentation
├── drivers
├── amlogic                          //Amlogic 芯片各种模块驱动
├── firmware
├── fs
├── include
├── init
├── ipc
├── kernel
├── lib
├── mm
├── net
├── samples
├── scripts
```

```
├── security
├── sound
├── soc/amlogic                    //Amlogic 声卡驱动
├── tools
├── usr
└── virt
```

Hardware 目录结构

hardware/amlogic/

```
├── aucpu_fw
├── audio                          //Amlogic audio HAL
├── boot_ctrl
├── camera                         //Amlogic camera HAL
├── consumerir
├── dumpstate
├── evs
├── fastboot
├── gralloc
├── hdmi_cec
├── health
├── hwcomposer
├── keymaster
├── LibAudio                           //Audio 播放和解码库
├── lights
├── media
├── media_modules                  //Decoder 驱动
├── memtrack
├── oemlock
├── power
├── screen_source
├── surround_view
├── tb_modules
├── thermal
├── tuner
├── tv
├── usb
└── wifi
```

Vendor 目录结构

vendor/amlogic/reference

```
├── aml_mp_sdk                     //Tsplayer 接口代码
├── apps                               //Amlogic APK 代码
│   ├── DroidLiveTvSettings
│   ├── DroidTvSettings
│   ├── LauncherCustomization
│   └── TvInput
├── external
│   ├── DTVKit                     //DTVKit 包
├── libdvr                             //DVR 代码
├── libsecdmx_release
├── prebuilt
│   ├── amazon
│   ├── ExoPlayer
│   ├── firmware
```

```
│   ├── kernel-modules
│   ├── LeanbackCustomizer
│   ├── livetv
│   ├── readlog
│   ├── scripts
│   └── SkiaPort
├── subtitle
│   └── subtitleserver          //字幕服务
└── tv
    ├── apps
    ├── frameworks
    └── tvserver
```

vendor/amlogic/common/

```
├── apps
│   ├── MicToggle
│   ├── Netflix
│   ├── Updater
│   └── VendorOverlay
├── codec2
├── dsp
├── external
├── frameworks
│   ├── av
│   ├── core                    //Amlogic java 层 API 支持包
│   └── services
│               ├── screencontrol    //录屏抓屏
│               ├── systemcontrol    //处理 HDMI 插拔和 HDCP 认证相关
├── gms
├── gpu
├── gpu-lib
├── hdcp                        //hdcp provision key 读取程序
├── interfaces
├── ir_tools
├── kernel
├── libdsm
├── mediahal_sdk                //Tsplayer 库
├── prebuilt
│   ├── dvb
│   ├── gpttool
│   ├── libmedia
│   ├── libmediadrm             //playready Widevine 相关库
│   ├── libstagefrighthw        //OMX 库
│   ├── multiwifi               //multi wifi 自动识别库
│   └── videofirm               //decoder 微码加载库
├── pre_submit_for_google       //针对 GOOGLE 原始代码的补丁
├── provision                   //provision 写 key 模块
├── scripts
├── system
├── tdk_linuxdriver             //optee 驱动
├── tdk_v3                      //芯片用的 secureos 包
├── tools
└── wifi_bt                     //WIFI/BT 用到的 firmware 和编译 Makefile
```

Common: （项目定制部分）

1) 内核配置项

common/arch/arm/configs/ #32 位内核

common/arch/arm64/configs/ #64 位内核

2) dts 配置

common/arch/arm64/boot/dts/amlogic/s4_*.dts #s4 不同公板配置

common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi #s4 公共配置

3) 常用驱动

audio pin mux 配置参考:

common/sound/soc/amlogic/auge/pinctrl/pctrl-audio.c

常用 pin control 配置参考：

common/drivers/pinctrl/meson/pinctrl-meson-s4.c

Device （项目定制部分）

device/amlogic/oppen

1) 编译配置（指定内核编译配置文件、内核 dts 文件、分 32bit 和 64bit）

device/amlogic/oppen/build.config.meson.arm.trunk

device/amlogic/oppen/build.config.meson.arm64.trunk

2) wifi 配置

device/amlogic/oppen/wifibt.build.config.trunk.mk

配置 MULTI_WIFI := true 时，表示编译所有支持的 WiFi

默认 MULTI_WIFI 为 false，按如下配置指定 WiFi：

CONFIG_WIFI_MODULES ?= qca6174 ap6398s w1 #指定 wifi

支持 WiFi 列表见：

vendor/amlogic/common/wifi_bt/wifi/configs/wifi.mk

3) BT 配置

device/amlogic/oppen/oppen.mk

BOARD_HAVE_BLUETOOTH := true #是否支持 bt

device/amlogic/oppen/wifibt.build.config.trunk.mk

CONFIG_BLUETOOTH_MODULES := multibt #multi：自动适配，也可以指定

支持 BT 列表见：

vendor/amlogic/common/wifi_bt/bluetooth/configs/bluetooth.mk

4) 自定义默认属性配置

device/amlogic/oppen/vendor_prop.mk

5) 预置 apk、预制 copy 文件配置

device/amlogic/common/products/mbox/product_mbox.mk

Bootloader （项目定制部分）

bootloader/uboot-repo/bl33/v2019/

1) uboot 编译配置

board\amlogic\defconfigs\s4_ap222_defconfig

board\amlogic\defconfigs\s4_ap229_defconfig

board\amlogic\defconfigs\s4_aq222_defconfig

2) board 配置头文件（默认 uboot 环境变量、kernel 启动参数等）

board/amlogic/configs/s4_ap222.h [s4_ap229.h, s4_aq222.h]

3) ddr 大小、频率配置

board/amlogic/s4_ap222/firmware/timing.c

board/amlogic/s4_ap229/firmware/timing.c

board/amlogic/s4_aq222/firmware/timing.c

vendor/amlogic/common

1) autopatch，此目录下的 patch 在 lunch 之后自动 patch

文件名规则：目录分隔符用#代替，#xxxx 数字表示 patch 序号

2) wifi 配置文件及 firmware

vendor/amlogic/common/wifi_bt/wifi

3) BT 配置文件及 firmware

vendor/amlogic/common/wifi_bt/bluetooth

4) Amlogic app

vendor/amlogic/reference/apps

hardware/amlogic

1) audio 相关（dolby 解码，google voice 降噪）

hardware/amlogic/audio

2) 视频解码库

hardware/amlogic/media_modules

2.3 代码编译

这部分建议参考 SDK 对应的 Release note 文档。

Android_S_release_notes_vxxxxxxxxx-GTVS.pdf

2.3.1 Bootloader 编译

```
cd ${projroot}/bootloader/uboot-repo/
./mk s4_ap222 --avb2 --vab
cp build/u-boot.bin.signed ../../device/amlogic/oppen/bootloader.img
cp build/u-boot.bin.sd.bin.signed ../../device/amlogic/oppen/upgrade/
cp build/u-boot.bin.usb.signed ../../device/amlogic/oppen/upgrade/
```

参数说明：

- --avb2: enable avb
- --bl32:如果需要特殊的 BL32，可以用这个后加指定 BL32 路径来替换默认的 BL32
- --vab: use vab(virtual a/b) to match frameworks

2.3.2 Kernel 编译

编译 kernel：

```
cd ${projroot}/
./mk oppen -v 5.4 -t user
```

生成 boot.img：

```
make bootimage
```

Clean

```
./mk clean
```

2.3.3 Android 编译

```
cd ${projroot}/
source build/envsetup.sh
lunch oppen_gtv-user
make -j8
```

注意：编译 Android 的时候不会编译 Kernel 和 bootloader，如果这两部分有修改，先编译着两个部分，再编译 Android.

如果出现 source build/envsetup.sh lunch 命令报错，有可能是在 build 目录下面执行了

git clean -df 或者 git reset --hard，导致 build 目录下的一些链接文件被删除导致。

新下载的干净目录，应该是如下图 1 和图 2

图 1

```
abi
bazel
blueprint
build_abi.sh
build.config -> ../build.config
build.config.net_test
build.sh
buildspec.mk.default -> make/buildspec.mk.default
build_test.sh
build-tools
CleanSpec.mk -> make/CleanSpec.mk
core -> make/core
envsetup.sh -> make/envsetup.sh
.git -> ../.repo/projects/build.git
gki
hermetic
LICENSE
make
METADATA
MODULE_LICENSE_APACHE2
multi-switcher.sh
NOTICE -> LICENSE
pesto
_setup_env.sh
soong
static_analysis
target -> make/target
tools -> make/tools
upstream
```


图 2

```
Not currently on any branch.
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        typechange: envsetup.sh

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        CleanSpec.mk
        bazel/
        blueprint/
        buildspec.mk.default
        core
        make/
        pesto/
        soong/
        target
        tools
```

如果 source build/envsetup.sh lunch 命令报错，则需要重新 sync 下载代码，或者手动创建这些链接文件。见. repo\manifests\default. xml

```
<project name="platform/build" path="build/make" revision="3b1bf7f674c3c379f76d25a1d7f7a8dc1856cbd1" groups="pdk">
  <copyfile src="core/root.mk" dest="Makefile"/>
  <linkfile src="CleanSpec.mk" dest="build/CleanSpec.mk"/>
  <linkfile src="buildspec.mk.default" dest="build/buildspec.mk.default"/>
  <linkfile src="core" dest="build/core"/>
  <linkfile src="envsetup.sh" dest="build/envsetup.sh"/>
  <linkfile src="target" dest="build/target"/>
  <linkfile src="tools" dest="build/tools"/>
</project>
```

2.3.4 OTA 全量包与差分包编译

1.全量包编译

cd ${projroot}/

source build/envsetup.sh

lunch oppen_gtv-user

make otapackage

生成 ota 全量包路径

/out/target/product/oppen/oppen-ota-eng.xxxxx.zip

2.差分包制作

选择需要差分的两个 target，路径：

out/target/product/oppen_old/obj/PACKAGING/target_files_intermediates/oppen-target_files-eng.xxxxx.zip

制作产生差分包命令：

./build/tools/releasetools/ota_from_target_files -p out/host/linux-x86/ -i ${target_src} ${target_dst} ${target_increm}

2.3.5 单独分区编译及烧录

Partition name	make command	imgae name	burning ways
boot	make bootimage	boot.img	fastboot flash boot boot.img
logo	make logoimg	logo.img	fastboot flash logo logo.img
uboot	./mk s4_ap222 --avb2 --vab	u-boot.bin.signed	fastboot flash bootloader u-boot.bin.signed
odm_ext	make odm_ext_image	odm_ext.img	fastboot flash odm_ext odm_ext.img
vendor_boot	make vendorbootimage	vendor_boot.img	fastboot flash vendor_boot vendor_boot.img
system	make systemimage	system.img	fastboot flash system system.img
vendor	make vendorimage	vendor.img	fastboot flash vendor vendor.img
odm	make odmimage	odm.img	fastboot flash odm odm.img
product	make productimage	product.img	fastboot flash product product.img
system_ext	make systemextimage	system_ext.img	fastboot flash system_ext system_ext.img

2.4 烧录工具及使用

2.4.1 USB 烧录工具（烧录整个大包）

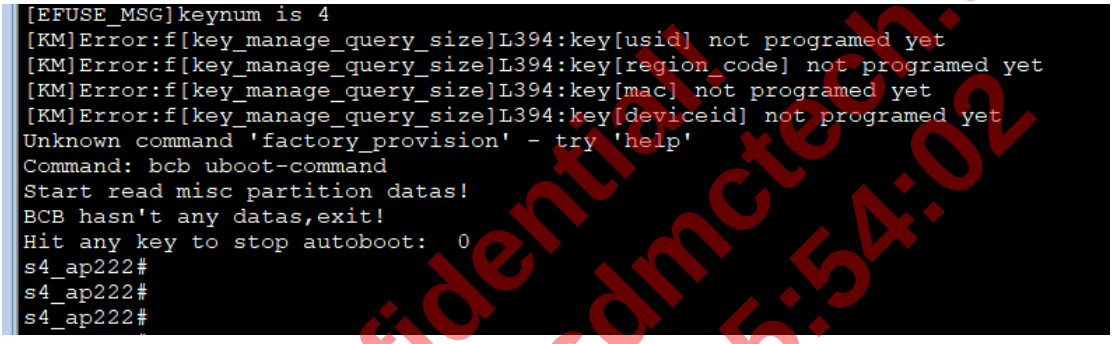
PC 端确保先安装 Aml_Burn_Tool_Setup.zip（新版本 v3.X）烧录工具软件。

打开烧录工具，“帮助”--> “使用说明”里有说明怎么烧录。简单介绍如下：

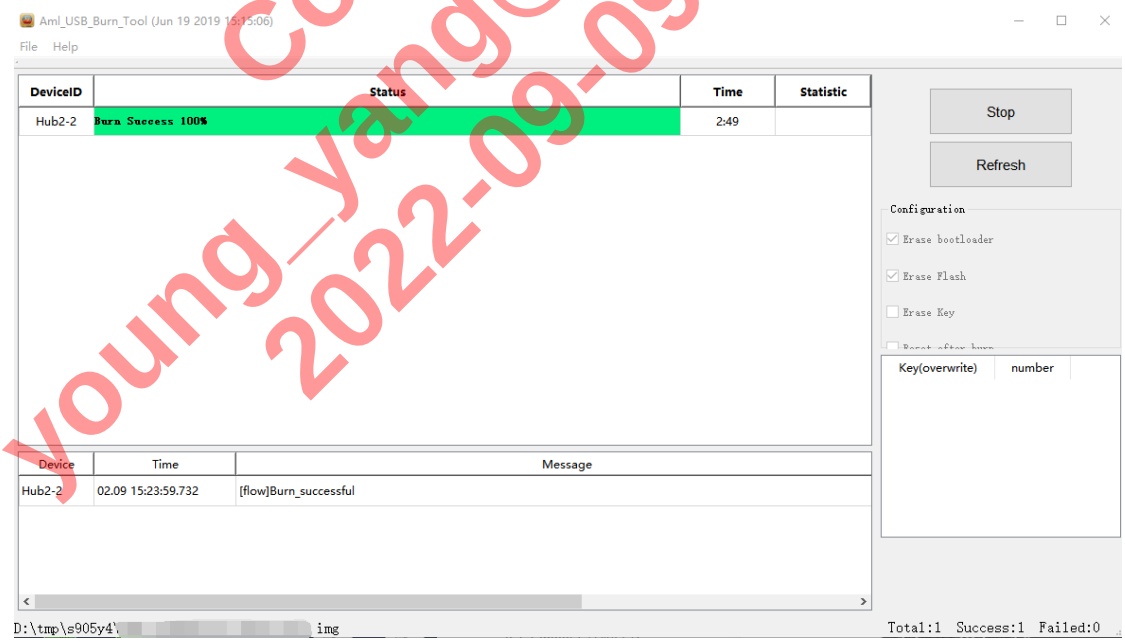
打开烧录工具，点击“设置”--> “导入镜像”-->选择编译生成的 aml_upgrade_package.img

USB 线连接 PC 和开发板，点击 start，开发板上电。

如果 flash 是空片，会自动开始烧录；如果 flash 已经烧录过，则先连接串口（波特率 921600）再上电，在刚启动的时候，串口中输入多次回车键，使得停在 uboot 命令行



然后输入 adnl 命令开始烧录，此时 PC 端烧录软件开始显示烧录进度。



待烧录进度 100%，烧录软件上点 Stop，再断电.

2.4.2 adnl 工具使用（烧录单独分区）

如果调试过程中，需要单独的替换某一个分区，可以通过 adnl 工具来替换，可以替换的分区有 bootloader, boot, logo, vendor_boot 等等，注意要先关闭 avb（Android Verified Boot）。例如以替换 boot.img 为例：

开发板连串口（波特率 921600）之后上电之后马上在串口中输入多次回车键使得进入 uboot 命令行。输入 adnl 命令进入烧录模式

```
s4_ap222#
s4_ap222# adnl
PHY2=00000000fe03a020,PHY3=00000000fe03a080
0x10 trim value=0x0000007f

USB RESET
SPEED ENUM
sof
```

在 window cmd 界面运行 adnl

cmd 窗口中输入烧录命令

adnl partition -p boot_a -f boot.img

更新 dts:

adnl partition -p vendor_boot_a -f vendor_boot.img

2.4.3 fastboot 工具使用（烧录单独分区）

Android 原始的 fastboot 来进行烧录单独的分区，以烧录 bootloader.img 为例，板子重新上电时候串口调试工具界面按住回车，停在 uboot 命令行下，输入 fastboot 命令进入烧录模式（在 android 下是通过 reboot bootloader 命令进入）：

```
s4_ap222# fastboot 0
serial num: 1234567890
PHY2=00000000fe03a020,PHY3=00000000fe03a080
0x10 trim value=0x0000007f

USB RESET
SPEED ENUM
```

在 window cmd 界面运行 fastboot 命令：

fastboot flashing unlock

fastboot flashing unlock_critical

fastboot flash bootloader bootloader.img

fastboot 重启命令(windows cmd 命令行输入)：

fastboot reboot

fastboot 在 uboot 下支持烧录的物理分区有：bootloader, boot, logo, vendor_boot

其他逻辑分区需要进入 recovery 中 fastbootd 模式烧录：system, system_ext, vendor,odm, product。可以在 recovery 中点 enter fastboot 菜单进入，或者在 android 下面通过 reboot fastboot 命令进入。

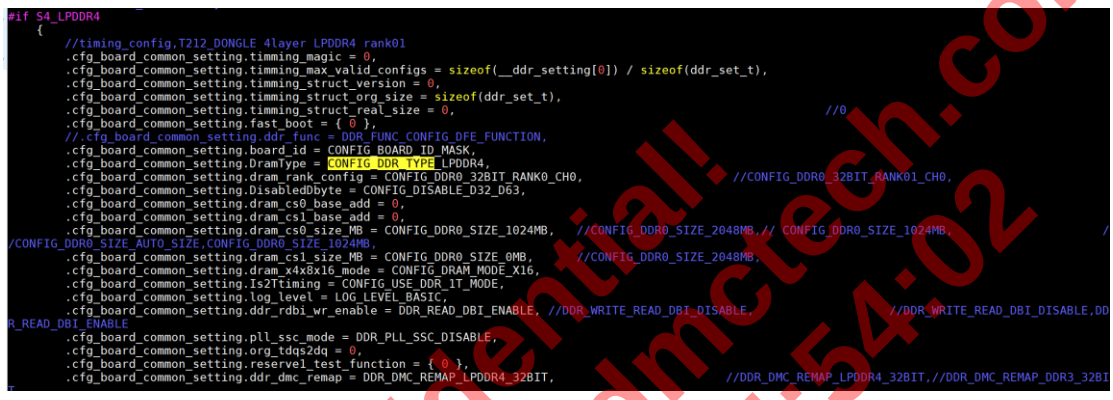
3. 常用外设配置

3.1.1 DDR 配置

根据项目实际 DDR 硬件设计，配置 DDR 类型、RANK、容量、频率。

内存颗粒（SDRAM）一般没有 64bits 数据线的，大多为 4bits，8bits 或 16bits。为了凑够 CPU 访问所需的 64bits，需要数据位扩展。如果每个颗粒是 8bits 位宽，那么就需要 8 个颗粒凑在一起，这八个颗粒组成一组，称为 rank。

一般项目 DDR 配置,以 S4 平台 S905Y4 LPDDR4 示例：



```
#if S4_LPDDR4
    {
        //timing_config,T212_DONGLE 4layer LPDDR4 rank01
        .cfg_board_common_setting.timming_magic = 0,
        .cfg_board_common_setting.timming_max_valid_configs = sizeof(__ddr_setting[0]) / sizeof(ddr_set_t),
        .cfg_board_common_setting.timming_struct_version = 0,
        .cfg_board_common_setting.timming_struct_org_size = sizeof(ddr_set_t),
        .cfg_board_common_setting.timming_struct_real_size = 0,                                    //0
        .cfg_board_common_setting.fast_boot = { 0 },
        //.cfg_board_common_setting.ddr_func = DDR_FUNC_CONFIG_DFE_FUNCTION,
        .cfg_board_common_setting.board_id = CONFIG_BOARD_ID_MASK,
        .cfg_board_common_setting.DramType = CONFIG_DDR_TYPE_LPDDR4,
        .cfg_board_common_setting.dram_rank_config = CONFIG_DDR0_32BIT_RANK0_CH0,                   //CONFIG_DDR0_32BIT_RANK01_CH0,
        .cfg_board_common_setting.DisabledDbyte = CONFIG_DISABLE_D32_D63,
        .cfg_board_common_setting.dram_cs0_base_add = 0,
        .cfg_board_common_setting.dram_cs1_base_add = 0,
        .cfg_board_common_setting.dram_cs0_size_MB = CONFIG_DDR0_SIZE_1024MB,   //CONFIG_DDR0_SIZE_2048MB,// CONFIG_DDR0_SIZE_1024MB,                /
/CONFIG_DDR0_SIZE_AUTO_SIZE,CONFIG_DDR0_SIZE_1024MB,
        .cfg_board_common_setting.dram_cs1_size_MB = CONFIG_DDR0_SIZE_0MB,      //CONFIG_DDR0_SIZE_2048MB,
        .cfg_board_common_setting.dram_x4x8x16_mode = CONFIG_DRAM_MODE_X16,
        .cfg_board_common_setting.Is2Ttiming = CONFIG_USE_DDR_1T_MODE,
        .cfg_board_common_setting.log_level = LOG_LEVEL_BASIC,
        .cfg_board_common_setting.ddr_rdbi_wr_enable = DDR_READ_DBI_ENABLE, //DDR_WRITE_READ_DBI_DISABLE,                  //DDR_WRITE_READ_DBI_DISABLE,DD
R_READ_DBI_ENABLE
        .cfg_board_common_setting.pll_ssc_mode = DDR_PLL_SSC_DISABLE,
        .cfg_board_common_setting.org_tdqs2dq = 0,
        .cfg_board_common_setting.reserve1_test_function = { 0 },
        .cfg_board_common_setting.ddr_dmc_remap = DDR_DMC_REMAP_LPDDR4_32BIT,                      //DDR_DMC_REMAP_LPDDR4_32BIT,//DDR_DMC_REMAP_DDR3_32BI
```

bootloader/uboot-repo/bl33/v2019/board/amlogic/s4_ap222/firmware/timing.c

3.1.2 DDR 参数选择

公板参数控制宏定义如下

#define S4_LPDDR4 1 //use for 1rank lpddr4

#define S4_LPDDR4_DONGLE_LAYER_4 1 //use for 1rank lpddr4

//#define S4_LPDDR4_DONGLE_LAYER_6 1 //use for 1rank lpddr4

//#define S4_LPDDR4_2RANK 1

#define S4_DDR4_2RANK 1

//#define S4_DDR4_1RANK 1

//#define S4_DDR3 1

结构体数组__ddr_setting 一般包含 4 套以上参数，每个元素使用大括号{}括起来，每个元素表示一套参数，每套参数会以轮询的方式从上到下进行尝试：

ddr_set_t __ddr_setting[] __attribute__ ((section(".ddr_param"))) = {

3.1.3 DDR 类型配置

公板 DDR 类型设置

.cfg_board_common_setting.DramType = CONFIG_DDR_TYPE_DDR4

bootloader/uboot-repo/bl33/v2019/arch/arm/include/asm/arch-s4/ddr_define.h 类型定义

```
#define CONFIG_DDR_TYPE_DDR3                    0
#define CONFIG_DDR_TYPE_DDR4                    1
#define CONFIG_DDR_TYPE_LPDDR4                  2
#define CONFIG_DDR_TYPE_LPDDR3                  3
#define CONFIG_DDR_TYPE_LPDDR2                  4
//#define CONFIG_DDR_TYPE_LPDDR4X               5
```

3.1.4 DDR Rank 配置

公板 RANK 设置

```
.cfg_board_common_setting.dram_rank_config = CONFIG_DDR0_32BIT_RANK0_CH0
```

dram_rank_config	说明	DDR3	DDR4	LPDDR3	LPDDR4
CONFIG_DDR0_16BIT_CH0	dram total bus width 16bit only use cs0	√	√	√	
CONFIG_DDR0_16BIT_RANK01_CH0	dram total bus width 16bit use cs0 cs1	√	√	√	
CONFIG_DDR0_32BIT_RANK0_CH0	dram total bus width 32bit use cs0	√	√	√	√
CONFIG_DDR0_32BIT_RANK01_CH0	dram total bus width 32bit use cs0 cs1	√	√	√	√
CONFIG_DDR0_32BIT_RANK01_CH01	only for lpddr4,dram total bus width 32bit use chanel a cs0 cs1 chanel b cs0 cs1,用在有两个 CS 的 LPDDR4				√
CONFIG_DDR0_32BIT_RANK0_CH01	only for lpddr4,dram total bus width 32bit use chanel a cs0 chanel b cs0，用在只有一个 CS 的 LPDDR4				√
CONFIG_DDR0_32BIT_16BIT_RANK0_CH0	dram total bus width 32bit only use cs0,but high address use 16bit mode，例如，接 2pcs DDR,总容量为 1.5GB,高 16bit 接 512MB，低 16bit 接 1GB	√	√		
CONFIG_DDR0_32BIT_16BIT_RANK01_CH0	保留				

3.1.5 DDR 频率配置

常用 DDR 频率 667 672 792 1176 1320（MHz）

公板 DDR 频率设置

.cfg_board_SI_setting_ps[0].DRAMFreq = 1176

DDR CS0/CS1 容量配置

bootloader/uboot-repo/bl33/v2019/arch/arm/include/asm/arch-s4/ddr_define.h 大小定义

```
#define     CONFIG_DDR0_SIZE_0MB          0
#define     CONFIG_DDR0_SIZE_128MB        128
#define     CONFIG_DDR0_SIZE_256MB        256
#define     CONFIG_DDR0_SIZE_512MB        512
#define     CONFIG_DDR0_SIZE_768MB        768
#define     CONFIG_DDR0_SIZE_1024MB       1024
#define     CONFIG_DDR0_SIZE_1536MB       1536
#define     CONFIG_DDR0_SIZE_2048MB       2048
#define     CONFIG_DDR0_SIZE_3072MB       3072
#define     CONFIG_DDR0_SIZE_4096MB       4096
#define     CONFIG_DDR0_SIZE_AUTO_SIZE        0xffff
#define     CONFIG_DDR1_SIZE_0MB          0
#define     CONFIG_DDR1_SIZE_128MB        128
#define     CONFIG_DDR1_SIZE_256MB        256
#define     CONFIG_DDR1_SIZE_512MB        512
#define     CONFIG_DDR1_SIZE_768MB        768
#define     CONFIG_DDR1_SIZE_1024MB       1024
#define     CONFIG_DDR1_SIZE_1536MB       1536
#define     CONFIG_DDR1_SIZE_2048MB       2048
#define     CONFIG_DDR1_SIZE_3072MB       3072
#define     CONFIG_DDR1_SIZE_4096MB       4096
#define     CONFIG_DDR1_SIZE_AUTO_SIZE        0xffff
```

公板 DDR 大小设置

.cfg_board_common_setting.dram_cs0_size_MB = CONFIG_DDR0_SIZE_1024MB,

.cfg_board_common_setting.dram_cs1_size_MB = CONFIG_DDR1_SIZE_1024MB,

3.1.6 Kernel DDR 容量配置

公板 DDR 大小配置

/common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222_drm.dts

```
    memory@00000000 {
        device_type = "memory";
        linux,usable-memory = <0x0 0x0 0x0 0x80000000>;
    };
```

1G 大小配置成 0x40000000

2G 大小配置成 0x80000000

4G 大小配置成 0xF0000000 即 3840M（总线限制）

修改完成后，在 uboot 启动的时候，会打印出当前 DDR 的相关信息，如大小、频率等

```
dram_type==DDR4
config==Rank01_32bit_ch0
DDR : DDR4 Rank01_32bit_ch0
DDR dramfreq=1176 MHz
Set ddr clk to 1176 MHz

package_info_value=103prepare training
CS0 size: 1024MB
CS1 size: 1024MB
Total size: 2048MB @ 1176MHz
DDR : 2048MB @1176MHz
cs0 DataBus test pass
cs1 DataBus test pass
cs0 AddrBus test pass
cs1 AddrBus test pass
```

3.1.7 DDR 调试命令

uboot 常用命令

1.动态设置 DDR 频率(单次生效.重启后恢复默认)

g12_d2pll 1176

g12_d2pll [DDR 频率]

kernel 常用命令

cat /sys/class/aml_ddr/freq

DDR clk = (freq * 2) MHz

```
console:/ # cat /sys/class/aml_ddr/freq
588 MHz
```

当前 DDR 频率为 1176 MHz

free -m

```
console:/ # free -m
              total        used        free      shared     buffers
Mem:            985         900          84           2           6
-/+ buffers/cache:          894          90
Swap:           255           6         249
```

3.2 EMMC 及分区大小配置

3.2.1 EMMC 接口配置

common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222_drm.dts

```
&sd_emmc_c {
    status = "okay";
    pinctrl-0 = <&emmc_pins>, <&emmc_ds_pins>;
    pinctrl-1 = <&emmc_clk_gate_pins>;
    pinctrl-names = "default", "clk-gate";

    bus-width = <8>;
    cap-mmc-highspeed;
    mmc-ddr-1_8v;
    mmc-hs200-1_8v;
//  mmc-hs400-1_8v;
    max-frequency = <200000000>;
    non-removable;
    disable-wp;

//  mmc-pwrseq = <&emmc_pwrseq>;
//  vmmc-supply = <&vddao_3v3>;
//  vqmmc-supply = <&vddao_1v8>;
};
```

DTS 配置说明：

mmc-hs200-1_8v //开启 HS200

// mmc-hs400-1_8v; 注意开启 HS400 时 HS200 配置也需要打开

max-frequency = <200000000>; //EMMC 频率

EMMC 公共配置 common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
sd_emmc_c: mmc@fe08c000 {
    compatible = "amlogic,meson-axg-mmc";
    reg = <0x0 0xfe08c000 0x0 0x800>,
        <0x0 0xfe000168 0x0 0x4>,
        <0x0 0xfe004000 0x0 0x4>;
    interrupts = <GIC_SPI 178 IRQ_TYPE_EDGE_RISING>;
    status = "disabled";
    clocks = <&clkc CLKID_NAND>,
        <&clkc CLKID_SD_EMMC_C_CLK_MUX>,
         <&clkc CLKID_SD_EMMC_C_CLK>,
         <&xtal>,
         <&clkc CLKID_HIFI_PLL>,
         <&clkc CLKID_HIFI_PLL>;
    clock-names = "core", "mux0", "mux1",
            "clkin0", "clkin1", "clkin2";
    card_type = <1>;
    ignore_desc_busy;
    mmc_debug_flag;
    tx_delay = <10>;
    src_clk_rate = <1152000000>;
//  resets = <&reset RESET_SD_EMMC_C>;
};
```

```
emmc_pins: emmc_pins {
    mux-0 {
        groups = "emmc_nand_d0",
            "emmc_nand_d1",
            "emmc_nand_d2",
            "emmc_nand_d3",
            "emmc_nand_d4",
            "emmc_nand_d5",
            "emmc_nand_d6",
            "emmc_nand_d7",
            "emmc_cmd";
        function = "emmc";
        bias-pull-up;
        drive-strength-microamp = <4000>;
    };

    mux-1 {
        groups = "emmc_clk";
        function = "emmc";
        bias-pull-up;
        drive-strength-microamp = <4000>;
    };
};
```

```
emmc_ds_pins: emmc_ds_pins {
    mux {
        groups = "emmc_nand_ds";
        function = "emmc";
        bias-pull-down;
        drive-strength-microamp = <4000>;
    };
};

emmc_clk_gate_pins: emmc_clk_gate_pins {
    mux {
        groups = "GPIOB_8";
        function = "gpio_periphs";
        bias-pull-down;
        drive-strength-microamp = <4000>;
    };
};
```

3.2.2 EMMC 分区配置

3.2.2.1 GPT 分区表

Android S 上新项目默认使用 GPT 分区表(GPT 分区表是标准的分区表)

分区表配置文件在 device/amlogic/oppen/part_table.txt

通过 GPT 工具 makegpt 生成 gpt.bin，并打包在 bootloader.img

单独编译 GPT

```
rm out/target/product/oppen/gpt.bin
make gptbin -j12
```

可以用 makegpt 工具查看 GPT 分区表，如下

```
                                        /s-hybrid$ ./out/host/linux-x86/bin/makegpt -v 4 --read ./out/target/product/oppen/gpt.bin
read name:./out/target/product/oppen/gpt.bin, gpt bin size: 34304
read a gpt binary to get the partition tables
ptn    start block     end block         size          attr         name
---- ------------- ------------- ------------ ------------------ ---------
#001         73728        204799          64M 0x0000000000000000 reserved
#002        221184        237567           8M 0x0000000000000000 env
#003        270336        274431           2M 0x0001000000000000 frp
#004        290816        307199           8M 0x0011000000000000 factory
#005        309248        358399          24M 0x0001000000000000 vendor_boot_a
#006        360448        409599          24M 0x0001000000000000 vendor_boot_b
#007        411648        419839           4M 0x0001000000000000 bootloader_a
#008        421888        430079           4M 0x0001000000000000 bootloader_b
#009        432128        497663          32M 0x0001000000000000 tee
#010        499712        516095           8M 0x0001000000000000 logo
#011        518144        522239           2M 0x0001000000000000 misc
#012        524288        528383           2M 0x0001000000000000 dtbo_a
#013        530432        534527           2M 0x0001000000000000 dtbo_b
#014        536576        552959           8M 0x0002000000000000 cri_data
#015        555008        587775          16M 0x0002000000000000 param
#016        589824        622591          16M 0x0001000000000000 odm_ext_a
#017        624640        657407          16M 0x0001000000000000 odm_ext_b
#018        659456        724991          32M 0x0001000000000000 oem_a
#019        727040        792575          32M 0x0001000000000000 oem_b
#020        794624        925695          64M 0x0001000000000000 boot_a
#021        927744       1058815          64M 0x0001000000000000 boot_b
#022       1060864       1093631          16M 0x0001000000000000 rsv
#023       1095680       1128447          16M 0x0001000000000000 metadata
#024       1130496       1134591           2M 0x0001000000000000 vbmeta_a
#025       1136640       1140735           2M 0x0001000000000000 vbmeta_b
#026       1142784       1146879           2M 0x0001000000000000 vbmeta_system_a
#027       1148928       1153023           2M 0x0001000000000000 vbmeta_system_b
#028       1155072       4841471           1G 0x0001000000000000 super
#029       4843520      33554397          13G 0x0004000000000000 userdata
---- ------------- ------------- ------------ ------------------ ---------
```

注意：1）对于 Andorid S 的 out 目录下的 bootloader.img，与单独编译 uboot 后在 device/amlogic/oppen 下的 bootloader.img 是有区别的，device 下的并不包含 gpt.bin，而 out 目录下才包含 gpt.bin。

2）当前版本(android_s_release_notes_v20220805-GTVS-Candidate-Release)用 fasboot 命令烧录 bootloader(fastboot flash bootloader bootloader.img)并不会更新 GPT 分区，需要如下命令单独烧录 gpt.bin：

```
fastboot erase gpt
fastboot flash gpt gpt.bin
```

3.2.2.2 如何添加新分区

在 Android S 版本新增客户自定义分区，由于 Android S 默认是 AB 系统的，所以，新增的分区，如果需要 OTA 更新，那么新增的分区，必须是 AB 的

新增自定义分区 cas 的步骤如下：

（1）**分区表修改**（Android S 使用的是 gpt 分区表，不是定义在 dtsi 中）

device/amlogic/oppen

```
--- a/part_table.txt
+++ b/part_table.txt
@@ -28,4 +28,6 @@ vbmeta_b,-,2M,1M,0x0001
 vbmeta_system_a,-,2M,1M,0x0001
 vbmeta_system_b,-,2M,1M,0x0001
 super,-,1800M,1M,0x0001
+cas_a,-,16M,1M,0x0002
+cas_b,-,16M,1M,0x0002
  userdata,-,-,1M,0x0004
```

（2）**新增分区的挂载参数：**

device/amlogic/oppen/fstab.ab_oem.amlogic

（只读）

```
+/dev/block/by-
name/cas       /cas          ext4     ro,barrier=1          wait,slotselect,
first_stage_mount
```

（读写）

```
+/dev/block/by-
name/cas       /cas          ext4     noatime,nosuid,nodev,nodelalloc,nombl
k_io_submit,errors=panic      wait,slotselect,first_stage_mount
```

（3）新增分区的打包烧录配置：

device/amlogic/oppen/upgrade/aml_upgrade_package_AB_vendor_boot.conf

```
+file="cas.img"      main_type="PARTITION"          sub_type="cas_a"
```

（4）新增 cas 分区配置：

google 配置了自定义分区的的方法，我们按照要求适配即可。

4.1 新增 device/amlogic/oppen/cas 目录

4.2 cas 目录下新建 cas_dict.txt 内容如下：

```
skip_fsck=true
partition_name=cas    （分区名）
```

4.3 cas 目录下新建 cas.mk 内容如下：

```
CUSTOM_IMAGE_MOUNT_POINT :=  cas                          （分区名）
CUSTOM_IMAGE_PARTITION_SIZE :=  16777216            （分区大小）
CUSTOM_IMAGE_FILE_SYSTEM_TYPE := ext4
CUSTOM_IMAGE_DICT_FILE := device/amlogic/oppen/cas/cas_dict.txt
```

```
CUSTOM_IMAGE_SELINUX := true
CUSTOM_IMAGE_COPY_FILES := device/amlogic/oppen/cas/1.txt:1.txt （需要预烧录的文件）
```

（5）BoardConfig.mk 配置新增分区

```
--- a/BoardConfig.mk
+++ b/BoardConfig.mk
@@ -295,6 +295,8 @@ TARGET_HOST_TOOL_PATH :=
vendor/amlogic/common/tools/host-tool
 #put here after all vendor configuration assigned evaluated.
 include device/amlogic/common/soong_config/soong_config.mk

+BOARD_ROOT_EXTRA_FOLDERS := cas
+
 ################################################################################
 #
 #                OEM Partitions based dynamic fingerprint
@@ -303,7 +305,8 @@ include
device/amlogic/common/soong_config/soong_config.mk
   ifeq ($(BOARD_USES_DYNAMIC_FINGERPRINT),true)
 #Building raw OEM images with "make custom_images"
 PRODUCT_CUSTOM_IMAGE_MAKEFILES := \
-       device/amlogic/oppen/oem/oem.mk
+       device/amlogic/oppen/oem/oem.mk \
+       device/amlogic/oppen/cas/cas.mk
```

（6）增加 selinux 权限

```
diff --git a/sepolicy/device.te b/sepolicy/device.te
index af175c7c..b1b889b4
--- a/sepolicy/device.te
+++ b/sepolicy/device.te
@@ -64,3 +64,4 @@ type galcore_device, dev_type;
 type dolby_fw_device, dev_type;
 type key_device, dev_type;
 type wifi_power_device, dev_type;
+type cas_block_device, dev_type;

diff --git a/sepolicy/file_contexts b/sepolicy/file_contexts
old mode 100644
new mode 100755
index 9d78e069..b5f4db66
--- a/sepolicy/file_contexts
+++ b/sepolicy/file_contexts
@@ -548,3 +548,6 @@
 /system/bin/kexec          u:object_r:init_exec:s0

 /dev/aml_dsm               u:object_r:aml_dsm_device:s0
```

```
+/dev/block/cas_a            u:object_r:cas_block_device:s0
+/dev/block/cas_b            u:object_r:cas_block_device:s0
+/cas(/.*)?                  u:object_r:cas_block_device:s0
```

```
diff --git a/sepolicy/update_engine.te b/sepolicy/update_engine.te
index 35f26112..6d320c3b
--- a/sepolicy/update_engine.te
+++ b/sepolicy/update_engine.te
@@ -40,3 +40,4 @@ allow update_engine system_server:binder transfer;
 allow update_engine sysfs:blk_file {read write getattr};
 allow update_engine rootfs:dir {getattr};
 allow update_engine system_block_fsck_device:blk_file { getattr ioctl open
read write };
+allow update_engine cas_block_device:blk_file rw_file_perms;
```

（7）增加 cas 分区的 ota 更新配置：

device/amlogic/common/core_amlogic.mk

```
+++ b/core_amlogic.mk
@@ -677,7 +677,8 @@ AB_OTA_PARTITIONS := \
        odm \
        dtbo \
        product \
-       bootloader
+       bootloader \
+       cas

   ifeq ($(BOARD_USES_ODM_EXTIMAGE), true)
  AB_OTA_PARTITIONS += \
```

device/amlogic/common/factory.mk

```
diff --git a/factory.mk b/factory.mk
index 782e20db..dbfb38a4
--- a/factory.mk
+++ b/factory.mk
@@ -93,6 +93,9 @@ ifeq ($(BOARD_USES_VBMETA_SYSTEM),true)
 BUILT_IMAGES += vbmeta_system.img
 endif

+BUILT_IMAGES += cas.img
+INSTALLED_RADIOIMAGE_TARGET += $(PRODUCT_OUT)/cas.img
+
 ifeq ($(strip $(HAS_BUILD_NUMBER)),false)
   # BUILD_NUMBER has a timestamp in it, which means that
   # it will change every time.  Pick a stable value.
```

（8）编译命令：

```
source build/envsetup.sh
lunch oppen-userdebug
```

//编译 cas 分区镜像

```
make custom_images
```

//编译 gpt，如果已经编译过，有修改的话，需要手动删除再编译

```
rm out/target/product/oppen/gpt.bin
```

```
make gptbin -j12
```

//编译整包

```
make otapackage -j12
```

常用命令：

查看分区信息

```
cat /proc/partitions
```

查看分区名字

```
ls -l /dev/block/by-name/
```

查看分区挂载

```
mount
```

3.3 Pinmux 配置

3.3.1 Uboot pinmux 配置

Uboot 通过写寄存器方式进行设置 pinmux 功能，使能某 pin 脚作为功能引脚，寄存器定义：bootloader/uboot-repo/bl33/v2019/arch/arm/include/asm/arch-s4/register.h

通过 readl 和 writel 接口对寄存器设置对应功能。

3.3.2 kernel pinmux 配置

Kernel 使用 pinmux 主要通过 dts 进行配置。主要两部分，第一 pimux 驱动的 dts 定义；第二其他驱动引用 pimux 驱动中定义。

Pinmux groups 与 fuction 定义：common/drivers/pinctrl/meson/pinctrl-meson-s4.c

S4 emmc pinmux 示例

Groups：

```
static const char * const emmc_groups[] = {
    "emmc_nand_d0", "emmc_nand_d1", "emmc_nand_d2", "emmc_nand_d3",
    "emmc_nand_d4", "emmc_nand_d5", "emmc_nand_d6", "emmc_nand_d7",
    "emmc_clk", "emmc_rst", "emmc_cmd", "emmc_nand_ds"
};
```

Function:

```
static struct meson_pmx_func meson_s4_periphs_functions[] __initdata = {
    FUNCTION(gpio_periphs),
    FUNCTION(i2c0),
    FUNCTION(i2c1),
    FUNCTION(i2c2),
    FUNCTION(i2c3),
    FUNCTION(i2c4),
    FUNCTION(uart_a),
    FUNCTION(uart_b),
    FUNCTION(uart_c),
    FUNCTION(uart_d),
    FUNCTION(uart_e),
    FUNCTION(emmc),
    FUNCTION(nand),
```

```
/* BANK_B func1 */
static const unsigned int emmc_nand_d0_pins[]       = { GPIOB_0 };
static const unsigned int emmc_nand_d1_pins[]       = { GPIOB_1 };
static const unsigned int emmc_nand_d2_pins[]       = { GPIOB_2 };
static const unsigned int emmc_nand_d3_pins[]       = { GPIOB_3 };
static const unsigned int emmc_nand_d4_pins[]       = { GPIOB_4 };
static const unsigned int emmc_nand_d5_pins[]       = { GPIOB_5 };
static const unsigned int emmc_nand_d6_pins[]       = { GPIOB_6 };
static const unsigned int emmc_nand_d7_pins[]       = { GPIOB_7 };
static const unsigned int emmc_clk_pins[]       = { GPIOB_8 };
static const unsigned int emmc_rst_pins[]       = { GPIOB_9 };
static const unsigned int emmc_cmd_pins[]       = { GPIOB_10 };
static const unsigned int emmc_nand_ds_pins[]       = { GPIOB_11 };
```

S4 905Y4 DTS：common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
emmc_pins: emmc_pins {
    mux-0 {
        groups = "emmc_nand_d0",
                 "emmc_nand_d1",
                 "emmc_nand_d2",
                 "emmc_nand_d3",
                 "emmc_nand_d4",
                 "emmc_nand_d5",
                 "emmc_nand_d6",
                 "emmc_nand_d7",
                 "emmc_cmd";
        function = "emmc";
        bias-pull-up;
        drive-strength-microamp = <4000>;
    };

    mux-1 {
        groups = "emmc_clk";
        function = "emmc";
        bias-pull-up;
        drive-strength-microamp = <4000>;
    };
};
```

3.3.3 pinmux 调试命令

查看当前 pin 脚使用的 pinmux 功能

注：Android S 版本需要在 uboot 下 Disable Selinux 后才能访问 debug 下的文件

setenv EnableSelinux permissive

saveenv

cat /sys/kernel/debug/pinctrl/fe000000.apb4\:pinctrl@4000-pinctrl-meson/pinmux-pins

```
pin 22 (GPIOE_0): UNCLAIMED
pin 23 (GPIOE_1): UNCLAIMED
pin 0 (GPIOB_0): device fe08c000.mmc function emmc group emmc_nand_d0
pin 1 (GPIOB_1): device fe08c000.mmc function emmc group emmc_nand_d1
pin 2 (GPIOB_2): device fe08c000.mmc function emmc group emmc_nand_d2
pin 3 (GPIOB_3): device fe08c000.mmc function emmc group emmc_nand_d3
pin 4 (GPIOB_4): device fe08c000.mmc function emmc group emmc_nand_d4
pin 5 (GPIOB_5): device fe08c000.mmc function emmc group emmc_nand_d5
pin 6 (GPIOB_6): device fe08c000.mmc function emmc group emmc_nand_d6
pin 7 (GPIOB_7): device fe08c000.mmc function emmc group emmc_nand_d7
pin 8 (GPIOB_8): device fe08c000.mmc function emmc group emmc_clk
pin 9 (GPIOB_9): UNCLAIMED
pin 10 (GPIOB_10): device fe08c000.mmc function emmc group emmc_cmd
pin 11 (GPIOB_11): device fe08c000.mmc function emmc group emmc_nand_ds
pin 12 (GPIOB_12): UNCLAIMED
```

UNCLAIMED 表示此 GPIO 口未被申请使用

查看当前 pin 脚 pimux 功能对应的 fuction

cat /sys/kernel/debug/pinctrl/fe000000.apb4\:pinctrl@4000-pinctrl-meson/pinmux-functions

```
function: emmc, groups = [ emmc_nand_d0 emmc_nand_d1 emmc_nand_d2 emmc_nand_d3 emmc_nand_d4 emmc_nand_d5 emmc_nand_d6 emmc_nand_d7 emmc_clk emmc_rst emmc_cmd emmc_nand_ds ]
```

查看当前 pin 脚 pimux 功能对应的 group

cat /sys/kernel/debug/pinctrl/fe000000.apb4\:pinctrl@4000-pinctrl-meson/pingroups

```
group: emmc_nand_d0
pin 0 (GPIOB_0)

group: emmc_nand_d1
pin 1 (GPIOB_1)

group: emmc_nand_d2
pin 2 (GPIOB_2)

group: emmc_nand_d3
pin 3 (GPIOB_3)

group: emmc_nand_d4
pin 4 (GPIOB_4)

group: emmc_nand_d5
pin 5 (GPIOB_5)

group: emmc_nand_d6
pin 6 (GPIOB_6)

group: emmc_nand_d7
pin 7 (GPIOB_7)

group: emmc_clk
pin 8 (GPIOB_8)

group: emmc_rst
pin 9 (GPIOB_9)

group: emmc_cmd
pin 10 (GPIOB_10)

group: emmc_nand_ds
pin 11 (GPIOB_11)
```


3.4 GPIO 配置

3.4.1 Uboot GPIO 配置

进入 uboot 命令行，命令“GPIO”可以访问 pin 脚 GPIO 功能

GPIOX_7 示例：

查看状态

```
gpio status -a
```

```
......
periphs-banks55: output: 1 [ ]
```

拉高

```
s4_ap222# gpio set GPIOX_7
gpio: pin GPIOX_7 (gpio 55) value is 1
```

拉低

```
s4_ap222# gpio clear GPIOX_7
gpio: pin GPIOX_7 (gpio 55) value is 0
```

查看输入

```
s4_ap222# gpio input GPIOX_7
gpio: pin GPIOX_7 (gpio 55) value is 0
```

3.4.2 Kernel GPIO 配置

标准 GPIO 驱动 sysfs 接口，需要打开内核配置选项：CONFIG_GPIO_SYSFS

common/arch/arm64/configs/meson64_a64_R_defconfig:399:CONFIG_GPIO_SYSFS=y

查看 S4 gpio 定义：common/include/dt-bindings/gpio/meson-s4-gpio.h

常用设置命令：

查看 GPIO ID

cat /sys/kernel/debug/gpio

查看到所有 gpio 序号及状态，以下操作示例：gpio-508 (GPIOZ_10)

```
console:/ # cat /sys/kernel/debug/gpio
gpiochip0: GPIOs 430-511, parent: platform/fe000000.apb4:pinctrl@4000, periphs-banks:
 gpio-430 (GPIOB_0             )
 gpio-431 (GPIOB_1             )
 gpio-432 (GPIOB_2             )
 gpio-433 (GPIOB_3             )
 gpio-434 (GPIOB_4             )
 gpio-435 (GPIOB_5             )
 gpio-436 (GPIOB_6             )
 gpio-437 (GPIOB_7             )
 gpio-438 (GPIOB_8             )
 gpio-439 (GPIOB_9             )
 gpio-440 (GPIOB_10            )
 gpio-441 (GPIOB_11            )
```

Export GPIO

```
echo 508 > /sys/class/gpio/export
```

执行后可以查看到该节点/sys/class/gpio/gpio508/

注意：若被占用则会提示申请失败

Unexport GPIO

```
echo 508 > /sys/class/gpio/unexport
```

设置 GPIO 输出

```
echo out > /sys/class/gpio/gpio508/direction
```

设置 GPIO 输出高电平

```
echo 1 > /sys/class/gpio/gpio508/value
```

设置 GPIO 输出低电平

```
echo 0 > /sys/class/gpio/gpio508/value
```

设置 GPIO 输入

```
echo in > /sys/class/gpio/gpio508/direction
```

查看 GPIO 输入电平

```
cat /sys/class/gpio/gpio508/value
```

3.5 I2C 配置

3.5.1 Kernel I2C 配置

I2C 总线的 DTS 配置，以 905Y4 示例：

公共配置 common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
i2c0: i2c@66000 {
    compatible = "amlogic,meson-i2c";
    reg = <0x0 0x66000 0x0 0x48>;
    interrupts = <GIC_SPI 160 IRQ_TYPE_EDGE_RISING>;
    #address-cells = <1>;
    #size-cells = <0>;
    clocks = <&clkc CLKID_I2C_M_A>;
    status = "disabled";
};

i2c1: i2c@68000 {
    compatible = "amlogic,meson-i2c";
    reg = <0x0 0x68000 0x0 0x48>;
    interrupts = <GIC_SPI 161 IRQ_TYPE_EDGE_RISING>;
    #address-cells = <1>;
    #size-cells = <0>;
    clocks = <&clkc CLKID_I2C_M_B>;
    status = "disabled";
};
```

支持 5 组 I2C controler，I2C 总线配置：

interrupts = <GIC_SPI 160 IRQ_TYPE_EDGE_RISING>; //中断相关设置、

clocks = <&clkc CLKID_I2C_M_A>; //I2C 控制器使用的 clock、

status = "disabled"; //I2C 总线的初始开关状态

S905Y4 项目 I2C 配置及总线设备
common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222.dts

```
aliases {
    serial0 = &uart_B;
    serial1 = &uart_A;
    serial2 = &uart_C;
    serial3 = &uart_D;
    serial4 = &uart_E;
    i2c0 = &i2c0;
    i2c1 = &i2c1;
    i2c2 = &i2c2;
    i2c3 = &i2c3;
    i2c4 = &i2c4;
    spi0 = &spifc;
    spi1 = &spicc0;
    tsensor0 = &p_tsensor;
};
```

```
&i2c1 {
    status = "okay";
    pinctrl-names="default";
    pinctrl-0=<&i2c1_pins2>;
    clock-frequency = <300000>; /* default 100k */

    tlc59116: tlc59116@60 {
        compatible = "amlogic,tlc59116_led";
        reg = <0x60>;
        status = "okay";

        led0 {
            default_colors = <0 0 0>;
            r_io_number = <0>;
            g_io_number = <10>;
            b_io_number = <5>;
        };
```


pinctrl-0=<&i2c1_pins2>; //I2C pinmux 引脚配置

clock-frequency = <300000>; //I2C 时钟频率设置

compatible = "amlogic,tlc59116_led"; //I2C 设备驱动名称

reg = <0x60>; //I2C 设备地址，注意为 7bit 地址，不包含读写位

status = "okay"; //使能当前 I2C 设备

3.5.2 I2C 调试工具

查看 i2c 控制器命令

```
i2cdetect -l
```

```
console:/ # i2cdetect -l
i2c-3   i2c             Meson I2C adapter                       I2C Adapter
i2c-1   i2c             Meson I2C adapter                       I2C Adapter
i2c-4   i2c             Meson I2C adapter                       I2C Adapter
```

查看 I2C-3 总线上挂设备地址

i2cdetect -y 3

```
console:/ # i2cdetect -y 3
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- 1a -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- 3a -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- 62 -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

查看 I2C-3 总线上设备地址为 0x62 寄存器值

i2cdump -f -y 3 0x62

```
console:/ # i2cdump -f -y 3 0x62
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 50 a1 2f 50 1f a3 fd 7a 0e 82 88 b7 d6 40 94 4a    P?/P???z?????@?J
10: 66 40 80 2b 6a 50 91 27 8f cc 21 10 97 ef f5 7f    f@?+jP?'??!?????
20: 4a 9b e0 e0 36 90 ab 97 c5 a8 f4 30 73 2f d2 2e    J???6??????0s/?.
30: 11 ed 30 21 15 0f 50 50 50 50 50 50 50 50 50 50    ??0!??PPPPPPPPPP
40: 50 50 50 50 50 50 50 50 50 50 50 50 50 50 50 50    PPPPPPPPPPPPPPPP
```

读取 I2C-3 总线上设备地址为 0x62 寄存器 0x16 值

i2cget -f -y 3 0x62 0x16

```
console:/ # i2cget -f -y 3 0x62 0x16
0x91
```

设置 I2C-3 总线上设备地址为 0x62 寄存器值为 0x1f

i2cset -f -y 3 0x62 0x16 0x1f b

```
console:/ # i2cset -f -y 3 0x62 0x16 0x1f b
console:/ #
console:/ # i2cget -f -y 3 0x62 0x16
0x1f
```

3.6 USB Device 与 Host 工作模式配置

dts 中关于 usb 的配置有如下几个，这里面关于 otg/host only/device only 的配置，大致参考如下说明配置接口

```
&dwc2_a {
    status = "okay";
    /** 0: normal, 1: otg+dwc3 host only, 2: otg+dwc3 device only*/
    controller-type = <3>;
};

&crg {
    status = "okay";
};

&usb2_phy_v2 {
    status = "okay";
    portnum = <2>;
};

&usb3_phy_v2 {
    status = "okay";
    portnum = <0>;
    otg = <1>;
    /*gpio-vbus-power = "GPIOH_6";*/
    /*gpios = <&gpio GPIOH_6 GPIO_ACTIVE_HIGH>;*/
};
```

Controller-type 的定义

common/include/linux/amlogic/usbtype.h

```
#define USB_NORMAL  0
#define USB_HOST_ONLY   1
#define USB_DEVICE_ONLY  2
#define USB_OTG          3
```

上述 code 就是配置的地方，目前我们代码里面默认配置的是 otg 模式，这种模式下，如果硬件支持 id pin 的自动检测的话；那么硬件会自动切换 host 和 device，这样，我们实时的接入 U 盘鼠标可以直接使用，接入 PC，也可以直接做 device 使用；

如果硬件不支持 id pin 的检测，根据目前我们软件配置，只能开机默认一种 host 或者 devide 工作模式，不能动态切；这种情况下，我们有一个 force device mode 的设置，就是强制设置 usb 工作在 device 模式的设置，这个已经默认设置成了非 0，就是可以默认 usb2.0 的口就是做 device；

```
module_param_named(otg_device, force_device_mode,
        bool, S_IRUGO | S_IWUSR);

static char otg_mode_string[2] = "0";
static int __init force_otg_mode(char *s)
{
    if (s != NULL)
        sprintf(otg_mode_string, "%s", s);
    if (strcmp(otg_mode_string, "0") == 0)
        force_device_mode = 0;
    else
        force_device_mode = 1;
    return 0;
}
__setup("otg_device=", force_otg_mode);
```

uboot 会将这个环境变量 otg_device=1 传给 kernel comdline，这样 usb 驱动起来的时候会去读取这个值，otg_device 不为 0，即表示 usb 2.0 otg 功能默认打开；这个口只能做 device 使用，不能在做 host；

```
[    0.000000@0]  Built 1 zonelists, mobility grouping on.  Total pages: 516096
[    0.000000@0]  Kernel command line: init=/init console=ttyS0,921600 no_console_suspend earlycon=aml-uart,0xfe07a000 ramoops.pstore_en=1 ramoops.record_size=0x8000 ramoops.console_size=0x4000 loop.max_part=4 loglevel=7 rootfstype=ramfs otg_device=1 logo=osd0,loaded,0x00300000 vout=1080p60hz,enable panel hdmimode=1080p60hz hdr_policy=0 hdr_priority= frac_rate_policy=1 hdmi_read_edid= cvbsmode=576cvbs osd_reverse=0 video_reverse=0 irq_check_en=0 androidboot.selinux=enforcing androidboot.firstboot=0 jtag=disable androidboot.hardware=amlogic androidboot.wificountrycode=US androidboot.serialno=1234567890 androidboot.rpmb_state=0x1 androidboot.force_normal_boot=1 androidboot.slot_suffix=_a androidboot.vbmeta.device=/dev/block/vbmeta androidboot.vbmeta.avb_version=1.1 androidboot.vbmeta.device_state=locked androidboot.vbmeta.hash_alg=sha256 androidboot.vbmeta.size=9984 androidboot.vbmeta.digest=dea49fa13a2f62f24006bdcb84357d1ba91ca1f29d0cf7057390e4921db2510c androidboo
[    0.000000@0]  Dentry cache hash table entries: 65536 (order: 7, 524288 bytes, linear)
```

如果客户需要将这个口设置成 host 的话，可以修改 uboot 下的这个环境变量：

setenv otg_device 0; saveenv; reset;

然后重新启动后，usb2.0 的这个口就默认做 host 了；

device\amlogic\oppen\BoardConfig.mk 中可以强制编译为 device 模式

```
177  ifneq ($(USE_USB_AS_HOST),true)
178  BOARD_KERNEL_CMDLINE += "otg_device=1"
179  endif
```

3.6.1 配置为 OTG

usb3_phy_v2 节点中的：otg = <1>;

dwc2_a 节点中的：controller-type = <3>;

这个也是目前代码里面的默认配置。

3.6.2 配置为 host only

不能再切换到 device；

usb3_phy_v2 节点中的：otg = <0>;

dwc2_a 节点中的：controller-type = <1>;

3.6.3 配置为 device only

不能再切换到 host

usb3_phy_v2 节点中的：otg = <0>;

dwc2_a 节点中的：controller-type = <2>;

如果硬件支持 usb3.0 的话，usb3_phy_v2 节点中的：portnum = <1>;

如果硬件不支持 usb3.0 的话，usb3_phy_v2 节点中的：portnum = <0>;

3.7 WIFI & BT 配置

3.7.1 WIFI 驱动配置

所有厂商 wifi 驱动代码放在 vendor/wifi_driver 目录下，以 amlogic W155S1 移植示例：

1、移植 Wifi 厂家提供的 W155S1 驱动代码

vendor/wifi_driver/amlogic/w1/wifi/

2、增加 modules 与 modules_install 两个 target,编译对应的驱动并且拷贝套开发板中。
vendor/wifi_driver/amlogic/w1/wifi/project_w1/vmac/Makefile

```
modules:
    perl ./create_version_file.pl
    @echo "UNO_GXL_SUB_P200=$(UNO_GXL_SUB_P200)"
    @echo "PROJECT = $(PROJ_NAME)"
    @echo "UNOENV=$(UNOENV) MAC_VER=$(MAC_VER)"
    @echo "EXTRA_CFLAGS=$(EXTRA_CFLAGS)"
    $(MAKE) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE) -C $(KERNEL_SRC) M=$(M) modules
ifneq (${OUT_DIR},)
    $(CROSS_COMPILE)strip --strip-unneeded ${OUT_DIR}/$(M)/$(TARGET).ko
else
    $(STRIP) -S $(TARGET).ko
endif

modules_install:
    @$(MAKE) INSTALL_MOD_STRIP=1 M=$(M) -C $(KERNEL_SRC) modules_install
    mkdir -p ${OUT_DIR}/../vendor_lib/modules
    cd ${OUT_DIR}/$(M)/; find -name "*.ko" -exec cp {} ${OUT_DIR}/../vendor_lib/modules/ \;
```

3、增加 wifi W155S1 驱动编译配置
vendor/amlogic/common/wifi_bt/wifi/configs/5_4/config.mk

```
WIFI_SUPPORT_DRIVERS += w1
w1_build ?= true
w1_modules ?= w1
w1_src_path ?= $(DRIVER_DIR)/amlogic/w1/wifi
w1_copy_path ?=
w1_build_path ?= project_w1/vmac
w1_args ?=
```

4、增加 wifi W155S1 配置文件拷贝

vendor/amlogic/common/wifi_bt/wifi/configs/wifi.mk

```
ifneq ($(filter w1,$(WIFI_MODULES)),)
ifeq (,$(wildcard vendor/wifi_driver/amlogic/w1/wifi/project_w1/vmac))
PRODUCT_COPY_FILES += $(call find-copy-subdir-files,*.txt,vendor/amlogic/common/wifi_bt/wifi/w1,$(TARGET_COPY_OUT_VENDOR)/etc/wifi/w1/)
else
PRODUCT_COPY_FILES += $(call find-copy-subdir-files,*.txt,vendor/wifi_driver/amlogic/w1/wifi/project_w1/vmac,$(TARGET_COPY_OUT_VENDOR)/etc/wifi/w1/)
endif
PRODUCT_COPY_FILES += vendor/amlogic/common/wifi_bt/wifi/w1/iwpriv:$(TARGET_COPY_OUT_VENDOR)/etc/wifi/w1/iwpriv
WIFI_HIDL_FEATURE_DUAL_INTERFACE := true
endif
```

5、增加 WIFI 模组信息，支持 multi WIFI 功能

vendor/amlogic/common/wifi_bt/wifi/wifi_info/wifi_info.cc

```
{"8888","0000","vlsicomm","/vendor/lib/modules/vlsicomm.ko","","amlwifi",0x0,"","",true,true},\
```

字段 1: 8888 为 SDIO WIFI 模组的 ID 值

字段 2: 0000 为 PCIE WIFI 的 ID 值, W155S1 为 SDIO wifi,故此处为 0

字段 3：vlsicomm 为 W155S1 驱动 vlsicomm.ko 名称

字段 4：/vendor/lib/modules/vlsicomm.ko 驱动加载完整路径

字段 5：wifi 模块加载参数为空

字段 6：amlwifi 为 wifi 名字，方便 log 中查看

字段 7：0x0 为 USB WIFI 模组的 ID 值，W155S1 为 SDIO wifi,故此处为 0

字段 8：wifi FW 完整路径，W155S1 无 FW 文件，故此处为空

字段 9：BT 驱动名称，W155S1 无 BT 驱动 KO 文件，故为空

字段 10：true 为支持蓝牙

字段 11：true 为 WIFI BT 共用电源

6、添加 S905Y4 平台支持 W155S1 WIFI 驱动配置

device/amlogic/oppen/wifibt.build.config.trunk.mk

```
##################################################
CONFIG_WIFI_MODULES ?= qca6174 ap6398s w1
```

那么客户也就可以直接将 WIFI_MODULE 配置成列表种的某一个，这样 make 的时候就会根据指定的模组去编译驱动，并拷贝指定模组的 config 及 fw:

```
318
319 ########################################################################## bcm43458
320 ifeq ($(WIFI_MODULE),bcm43458)
321 WIFI_DRIVER := bcm43458
322 WIFI_DRIVER_MODULE_PATH := /vendor/lib/modules/dhd.ko
323 WIFI_DRIVER_MODULE_NAME := dhd
324 WIFI_DRIVER_MODULE_ARG  := "firmware_path=/vendor/etc/wifi/43458/fw_bcm43455c0_ag.bin nvram_path=/vendor/etc/wifi/43458/nvram_43458.txt"
325 WIFI_DRIVER_FW_PATH_STA := /vendor/etc/wifi/43458/fw_bcm43455c0_ag.bin
326 WIFI_DRIVER_FW_PATH_AP  := /vendor/etc/wifi/43458/fw_bcm43455c0_ag_apsta.bin
327 WIFI_DRIVER_FW_PATH_P2P := /vendor/etc/wifi/43458/fw_bcm43455c0_ag_p2p.bin
328
329 BOARD_WLAN_DEVICE := bcmdhd
330 WIFI_DRIVER_FW_PATH_PARAM   := "/sys/module/dhd/parameters/firmware_path"
331
332 WPA_SUPPLICANT_VERSION := VER_0_8_X
333 BOARD_WPA_SUPPLICANT_DRIVER := NL80211
334 BOARD_WPA_SUPPLICANT_PRIVATE_LIB := lib_driver_cmd_bcmdhd
335 BOARD_HOSTAPD_DRIVER        := NL80211
336 BOARD_HOSTAPD_PRIVATE_LIB   := lib_driver_cmd_bcmdhd
337
338 PRODUCT_PACKAGES += \
339         43458/nvram_43458.txt \
340         43458/fw_bcm43455c0_ag.bin \
341       43458/fw_bcm43455c0_ag_apsta.bin \
342       43458/fw_bcm43455c0_ag_p2p.bin \
343         wl \
344     p2p_supplicant_overlay.conf \
345         dhd
```

3.7.2 BT 配置

所有厂商 BT 驱动放在 vendor/amlogic/common/wifi_bt/bluetooth,以 W155S1 移植示例:

1、移植 W155S1 BT 驱动

vendor/amlogic/common/wifi_bt/bluetooth/amlogic

2、添加 BT 驱动编译配置

vendor/amlogic/common/wifi_bt/bluetooth/configs/5_4/Makefile

modules:

```
    $(MAKE) -C $(KERNEL_SRC)/../vendor/amlogic/common/wifi_bt/bluetooth/amlogic/sdio_driver_bt KERNEL_SRC=$(KERNEL_SRC) M=../vendor/amlogic/common/wifi
_bt/bluetooth/amlogic/sdio_driver_bt O=$(O) CC=$(CC) HOSTCC=$(HOSTCC) LD=$(LD) NM=$(NV) OBJCOPY=$(OBJCOPY) SUBDIRS=$(KERNEL_SRC)/../vendor/amlogic/comm
on/wifi_bt/bluetooth/amlogic/sdio_driver_bt
```

modules_install:

```
    $(MAKE) -C $(KERNEL_SRC)/../vendor/amlogic/common/wifi_bt/bluetooth/amlogic/sdio_driver_bt M=../vendor/amlogic/common/wifi_bt/bluetooth/amlogic/sdi
o_driver_bt KERNEL_SRC=$(KERNEL_SRC) O=$(O) CC=$(CC) HOSTCC=$(HOSTCC) LD=$(LD) NM=$(NV) OBJCOPY=$(OBJCOPY) INSTALL_MOD_PATH=$(INSTALL_MOD_PATH) SUBDIRS
=$(KERNEL_SRC)/../vendor/amlogic/common/wifi_bt/bluetooth/amlogic/sdio_driver_bt modules_install
```

3、W155S1 BT 配置文件以及 libbt-vendor 编译配置

/vendor/amlogic/common/wifi_bt/bluetooth/configs/bluetooth.mk

```
ifeq ($(BLUETOOTH_MODULE), AMLBT)

#load aml mk
$(call inherit-product, vendor/amlogic/common/wifi_bt/bluetooth/amlogic/amlbt.mk )

PRODUCT_COPY_FILES += vendor/amlogic/common/wifi_bt/bluetooth/configs/init_rc/init.amlogic.amlbt.rc:$(TARGET_COPY_OUT_VENDOR)/etc/init/hw/init.amlogic.bluetooth.rc

PRODUCT_PACKAGES += libbt-vendor
endif
```

4、添加 S905Y4 平台支持 W155S1 WIFI 驱动配置

```
ifeq ($(BLUETOOTH_MODULE), multibt)

BOARD_HAVE_BLUETOOTH_MULTIBT := true
PRODUCT_PROPERTY_OVERRIDES += \
    ro.vendor.btmodule = multibt

$(call inherit-product, vendor/amlogic/common/wifi_bt/bluetooth/unisoc/libbt/unisocbt.mk )
$(call inherit-product, vendor/amlogic/common/wifi_bt/bluetooth/realtek/rtkbt/rtkbt.mk )
$(call inherit-product, vendor/amlogic/common/wifi_bt/bluetooth/mtk/mtkbt/mtkbt.mk )
$(call inherit-product, vendor/amlogic/common/wifi_bt/bluetooth/broadcom/bcmbt.mk )
$(call inherit-product, vendor/amlogic/common/wifi_bt/bluetooth/qualcomm/qcabt.mk )
$(call inherit-product, vendor/amlogic/common/wifi_bt/bluetooth/amlogic/amlbt.mk )

PRODUCT_COPY_FILES += vendor/amlogic/common/wifi_bt/bluetooth/configs/init_rc/init.amlogic.amlbt.rc:$(TARGET_COPY_OUT_VENDOR)/etc/init/hw/init.ap201.bluetooth.rc

PRODUCT_PACKAGES += \
    libbt-vendor_bcm \
    libbt-vendor_rtl \
    libbt-vendor_mtk \
    libbt-vendor_qca \
    libbt-vendor_uwe \
    libbluetooth_mtkbt \
    libbt-vendor_aml
endif
```

BOARD_HAVE_BLUETOOTH := true

BLUETOOTH_MODULE := multibt

include vendor/amlogic/common/wifi_bt/bluetooth/configs/bluetooth.mk

如果要平台支持 bt，那么 BOARD_HAVE_BLUETOOTH := true 一定要配置上，这个配置上后，编译的时候，才会将 Android 里面的 BT 相关的 properties 给置上，并且会将一些 BT 的配置文件给编译进去；这个配置其实是针对 Android 的，不是针对某一个模组的配置；

如果要指定 bt 模组，则配置 BLUETOOTH_MODULE := RTKBT，一般可选值为 BCMBT/MTKBT/RTKBT/QCABT；客户可以根据自己的硬件自行配置。

3.7.3 Wifi&BT kernel 配置

对于 wifi&bt 硬件配置，包含 wifi&BT 上下电驱动、wake 中断配置、SDIO 卡配置、32K 时钟配置。以 S905Y4 的 W155S1 示例：

WIFI & BT 上下电配置 common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
aml_bt: aml_bt {
    compatible = "amlogic, aml-bt";
    status = "disabled";
};

aml_wifi: aml_wifi {
    compatible = "amlogic, aml-wifi";
    status = "disabled";
    irq_trigger_type = "GPIO_IRQ_LOW";
    //dhd_static_buf; /* if use bcm wifi, config dhd_static_buf */
    //pinctrl-0 = <&pwm_e_pins>;
    //pinctrl-names = "default";
    pwm_config = <&wifi_pwm_conf>;
};
```

common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222_drm.dts

```
&aml_wifi{
    status = "okay";
    interrupt-gpios = <&gpio  GPIOX_7  GPIO_ACTIVE_HIGH>;
    power_on-gpios = <&gpio   GPIOX_6  GPIO_ACTIVE_HIGH>;
};

&aml_bt {
    status = "okay";
    reset-gpios = <&gpio    GPIOX_17 GPIO_ACTIVE_HIGH>;
    bt_en-gpios = <&gpio    GPIOX_17 GPIO_ACTIVE_HIGH>;
    btwakeup-gpios = <&gpio    GPIOX_18 GPIO_ACTIVE_HIGH>;
    hostwake-gpios = <&gpio GPIOX_19 GPIO_ACTIVE_HIGH>;
};
```

WIFI:

interrupt-gpios: WIFI 数据中断

power_on-gpios: WIFI 模组供电引脚控制

BT:

reset-gpios: wifi 模组供电控制

btwakeup-gpios：模组 BT 唤醒主控配置

hostwake-gpios: 主控唤醒模组 BT 配置

32K PWM 时钟配置 common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
wifi_pwm_conf:wifi_pwm_conf{
    pwm_channel1_conf {
        pwms = <&pwm_ef 0 30541 0>;
        duty-cycle = <15270>;
        times = <7>;
    };
    pwm_channel2_conf {
        pwms = <&pwm_ef 2 30500 0>;
        duty-cycle = <15250>;
        times = <10>;
    };
};
```

SDIO 卡配置 common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222_drm.dts

```
&sd_emmc_a {
    status = "okay";
    pinctrl-0 = <&sdio_pins>;
    pinctrl-1 = <&sdio_clk_gate_pins>;
    pinctrl-names = "default", "clk-gate";
    #address-cells = <1>;
    #size-cells = <0>;

    bus-width = <4>;
    cap-sd-highspeed;
    sd-uhs-sdr50;
    sd-uhs-sdr104;
    max-frequency = <200000000>;

    non-removable;
    disable-wp;

    //vmmc-supply = <&vddao_3v3>;
    //vqmmc-supply = <&vddio_ao1v8>;
```

3.7.4 WIFI&BT 调试命令

Wifi 上下电控制命令

上电：echo 1 > /sys/class/aml_wifi/power

下电：echo 2 > /sys/class/aml_wifi/power

查询网络设状态：ifconfig wlan0

wpa_cli 常用调试命令

搜索附近 wifi 网络: wpa_cli -i wlan0 scan

打印搜索 wifi 网络结果: wpa_cli -i wlan0 scan_result

添加一个网络连接: wpa_cli -i wlan0 add_network

设置 WIFI SSID : wpa_cli -i wlan0 set_network 1 ssid \"speakerteam_5G\"

设置 WIFI 密码 : wpa_cli -i wlan0 set_network 1 psk \"123123123\"

设置 WIFI 无密码 : wpa_cli -i wlan0 set_network 0 key_mgmt NONE

隐藏 SSID wpa_cli -i wlan0 set_network 0 scan_ssid 1

连接 wifif : wpa_cli -i wlan0 enable_network 1

保存配置: wpa_cli save

查询已添加网络 : wpa_cli -i wlan0 list_network

查询已连接 wifi 状态: wpa_cli -i wlan0 status

查询已连接 wifi SSID 值： iw wlan0 link

BT 上下电控制命令

上电：echo 0 > /sys/class/rfkill/rfkill0/state

下电：echo 1 > /sys/class/rfkill/rfkill0/state

查看 BT 状态

dumpsys bluetooth_manager

抓 btsnoop 命令

setprop persist.bluetooth.btsnoopdefaultmode null

3.8 GPIO Key 配置

3.8.1 Uboot gpio power key 配置

S905Y4 公板 GPIO power key 示例:

bootloader/uboot-repo/bl30/src_ao/demos/amlogic/n200/s4/s4_ap222/keypad.c

```
/*KEY ID*/
#define GPIO_KEY_ID_POWER    GPIOD_2
```

3.8.2 Kernel GPIO key 配置

Kernel GPIO key 驱动编译配置：common/arch/arm64/configs/meson64_a64_R_defconfig

CONFIG_AMLOGIC_INPUT_KEYBOARD=m

CONFIG_AMLOGIC_GPIO_KEY=m

S905Y4 公板 GPIO key 示例:
common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222.dts

```
gpio_keypad{
    compatible = "amlogic, gpio_keypad";
    status = "okay";
    scan_period = <20>;
    key_num = <1>;
    key_name = "bluetooth";
    key_code = <600>;
    key-gpios = <&gpio  GPIOD_2  GPIO_ACTIVE_HIGH>;
    detect_mode = <0>;/*0:polling mode, 1:irq mode*/
};
```

key_num = <1>; //GPIO 按键个数

key_code = <600>;//input 子系统上报的 linux 键值

key-gpios = <&gpio GPIOD_2 GPIO_ACTIVE_HIGH>; //GPIO 引脚使用

detect_mode = <0>; //检测模式选择 0：轮询方式 1：中断方式

注意事项:

1、GPIO 默认高电平，按下时接地，即抬起为 1，按下为 0

2、GPIO KEY 驱动不支持长按，由 Android 检测按下时间支持长按功能

3、驱动支持多个 GPIO 按键；驱动不支持组合按键，需要用户层检测实现

3.8.3 GPIO key 调试命令

查看 GPIO key 配置信息

cat /sys/class/gpio_keypad/table

```
console:/ # cat /sys/class/gpio_keypad/table
[0]: name = bluetooth                status = 1
```

查看 GPIO key 设备信息

cat /proc/bus/input/devices

```
I: Bus=0010 Vendor=0001 Product=0001 Version=0100
N: Name="gpio_keypad"
P: Phys=gpio_keypad/input0
S: Sysfs=/devices/platform/gpio_keypad/input/input3
U: Uniq=
H: Handlers=event3
B: PROP=0
B: EV=3
B: KEY=1000000 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

查看 input 输入事件

getevent

```
add device 4: /dev/input/event3
  name:     "gpio_keypad"
```

3.9 ADC Key 配置

3.9.1 以 S905Y4 硬件设计 ADC 采用 channel_0 的 ADC key 示例

3.9.2 Uboot ADC power key 配置

```
s4_ap222# saradc
saradc - saradc sub-system

Usage:
saradc saradc open <channel> <mode> - open a SARADC channel
       mode: 1:average 2:high precision 3:high resolution 4:decim filter
saradc close  - close the SARADC
saradc getval - get the value in current channel
saradc test   - test the SARADC by channel-7
saradc get_in_range <min> <max> - return 0 if current value in the range of current channel

s4_ap222# saradc open 0
SARADC mode is average
s4_ap222#
```

按下 ADC power 按键，然后执行 saradc getval 可以得到对应的采样值，采样值配置 AML_ADC_POWER_KEY_VAL，用作 adc 按键唤醒功能。

bootloader/uboot-repo/bl33/v2019/board/amlogic/configs/s4_ap222.h

```
--- a/board/amlogic/configs/s4_ap222.h
+++ b/board/amlogic/configs/s4_ap222.h
@@ -43,8 +43,8 @@
 #define AML_IR_REMOTE_POWER_UP_KEY_VAL9 0xFFFFFFFF

 /*config the default parameters for adc power key*/
-#define AML_ADC_POWER_KEY_CHAN   2  /*channel range: 0-7*/
-#define AML_ADC_POWER_KEY_VAL    0  /*sample value range: 0-1023*/
+#define AML_ADC_POWER_KEY_CHAN   0  /*channel range: 0-7*/
+#define AML_ADC_POWER_KEY_VAL    512  /*sample value range: 0-1023*/
```

3.9.3 kernel 下的 ADC key 配置

S905Y4 ADC 按键示例

Kernel ADC key 驱动编译配置：

common/arch/arm64/configs/meson64_a64_R_defconfig

CONFIG_MESON_SARADC=m

CONFIG_AMLOGIC_INPUT_KEYBOARD=m

CONFIG_AMLOGIC_ADC_KEYPADS=m

Kernel ADC key driver DTS 配置：

common\arch\arm64\boot\dts\amlogic\s4_s905y4_ap222_drm.dts

```
adc_keypad {
    compatible = "amlogic, adc_keypad";
    status = "okay";
    key_name = "standby", "vol+", "vol-";
    key_num = <3>;
    io-channels = <&saradc 0>;
    io-channel-names = "key-chan-0";
    key_chan = <0 0 0>;
    key_code = <116 115 114>;
    key_val = <20 910 630>; //val=voltage/1800mV*1023
    key_tolerance = <40 40 40>;
};
```

一般项目上可以修改的值：

io-channels = <&saradc 0>; // 配置 ADC 对应的 channel

key_name 方便阅读的按键名，无实际作用

key_num = <3>; //按键个数

key_code = <116 115 114>; //input 子系统上报 linux 键值

key_val = <20 910 630>; //电压转换值（按照公式计算 val=voltage/1800mV*1023）

key_tolerance = <40 40 40>; //误差范围，采样值在 key_val +/- 40 内都能识别

注意事项：

1、不支持长按

2、不支持组合按键

3.9.4 ADC key 调试命令

查看 ADC key 配置

cat /sys/class/adc_keypad/table

```
console:/ # cat /sys/class/adc_keypad/table
[1]: name=standby               code=116   channel=0   value=20    tolerance=40
[2]: name=vol+                  code=115   channel=0   value=910   tolerance=40
[3]: name=vol-                  code=114   channel=0   value=630   tolerance=40
```

动态修改或添加 ADC 按键命令

echo [name]:[code]:[channel]:[value]:[tolerance] > /sys/class/adc_keypad/table

示例：echo source:466:0:0:40 > /sys/class/adc_keypad/table

```
console:/ # cat /sys/class/adc_keypad/table
[1]: name=standby               code=116   channel=0   value=20    tolerance=40
[2]: name=vol+                  code=115   channel=0   value=910   tolerance=40
[3]: name=vol-                  code=114   channel=0   value=630   tolerance=40
[4]: name=source                code=466   channel=0   value=0     tolerance=40
```

删除所有 adc key

echo null > /sys/class/adc_keypad/table

Kernel 读取 channel0 ADC 电压值

cat /sys/bus/iio/devices/iio\:device0/in_voltage0_input

Uboot 读取 channel0 ADC 电压值

打开通道

saradc open 0 1

读取 adc 电压值

saradc getval

```
s4_ap222# saradc open 0 1
SARADC mode is average
s4_ap222# saradc getval
SARADC channel(0) is 1023.
```

3.10 平台遥控器配置

3.10.1 uboot IR power key 键值配置

支持 IR power 按键唤醒功能，S905Y4 示例，配置 power key 值：

bootloader/uboot-repo/bl30/src_ao/demos/amlogic/n200/s4/s4_ap222/power.c

```
static IRPowerKey_t prvPowerKeyList[] = {
    { 0xef10fe01, IR_NORMAL}, /* ref tv pwr */
    { 0xba45bd02, IR_NORMAL}, /* small ir pwr */
    { 0xef10fb04, IR_NORMAL}, /* old ref tv pwr */
    { 0xf20dfe01, IR_NORMAL},
    { 0xe51afb04, IR_NORMAL},
    { 0x3ac5bd02, IR_CUSTOM},
    {}
        /* add more */
};
```

如上图，最大支持 9 个 powerkey，不使用的 key 配置为 0xFFFFFFFF，为红外编码：

2 位键值反码+2 位键值码+4 位用户码

3.10.2 Kernel IR 配置

S905Y4 IR 遥控器示例：common\arch\arm64\boot\dts\amlogic\meson-s4.dtsi

```
ir: ir@8000 {
    compatible = "amlogic, meson-ir";
    reg = <0x0 0xfe084040 0x0 0xA4>,
        <0x0 0xfe084000 0x0 0x20>;
    status = "disable";
    protocol = <REMOTE_TYPE_NEC>;
    interrupts = <GIC_SPI 22 IRQ_TYPE_EDGE_RISING>;
    map = <&custom_maps>;
    max_frame_time = <200>;
};
```

status：okay/disable 是否使能遥控器

protocol：遥控器协议类型，定义：common/include/dt-bindings/input/meson_rc.h

```
#define     REMOTE_TYPE_UNKNOWN     0x00
#define     REMOTE_TYPE_NEC         0x01
#define     REMOTE_TYPE_DUOKAN      0x02
#define     REMOTE_TYPE_XMP_1       0x03
#define     REMOTE_TYPE_RC5         0x04
#define     REMOTE_TYPE_RC6         0x05
#define     REMOTE_TYPE_TOSHIBA     0x06
#define     REMOTE_TYPE_RCA         0x08
#define     REMOTE_TYPE_RCMM        0x0a
```

custon_mapsIR 遥控器键值配置，修改适配新的遥控器键值映射关系：

common/arch/arm64/boot/dts/amlogic/meson-ir-map.dtsi

```
custom_maps: custom_maps {
    mapnum = <4>;
    map0 = <&map_0>;
    map1 = <&map_1>;
    map2 = <&map_2>;
    map3 = <&map_3>;
    map_0: map_0{
        mapname = "amlogic-remote-1";
        customcode = <0xfb04>;
        release_delay = <80>;
        size  = <51>;    /*keymap size*/
        keymap = <REMOTE_KEY(0x47, KEY_0)
            REMOTE_KEY(0x13, KEY_1)
            REMOTE_KEY(0x10, KEY_2)
            REMOTE_KEY(0x11, KEY_3)
            REMOTE_KEY(0x0F, KEY_4)
```

默认支持 4 种 amlogic 遥控器，示例新增 map_3 IR 遥控器配置：

1、修改 mapnum = <4>;

2、参考 map_0 新增 map_3 配置项

3、在 map_3 中根据遥控器键值修改用户码 customcode = <0x7788>;

4、修改按键个数为 38，size = <38>;

5、修改 map_3 中遥控器键值与 linux 功能键值映射关系；REMOTE_KEY(0x47, KEY_HOME) 代表遥控器 0x47 键值映射为 linux home 按键功能键值

注意：配置错误的时候，打开 IR 的调试可以看到如下打印

```
console:/ # [  305.999088@1]  meson-ir:framecode=0xff00fe06
[  305.999202@1]  meson-ir:receive scancode=0x0
[  305.999557@1]  meson-ir fe084040.ir: invalid custom:0xff00fe06
[  306.118912@1]  meson-ir:framecode=0x0
[  306.118938@1]  meson-ir:receive repeat
```

3.10.3 Android IR 配置

IR 遥控对应的 keylayout 文件：

device/amlogic/common/products/mbox/Vendor_0001_Product_0001.kl

映射顺序：

dts: 遥控器物理键值 --> dts: linux 键值 --> kl: linux 键值 --> kl: android 键值

举例：

customcode = <0xfe01>;中 REMOTE_KEY(0x1d, KEY_ENTER)

KEY_ENTER linux 键值为 28

Vendor_0001_Product_0001.kl 中 28 对应 DPAD_CENTER

3.10.4 IR 调试命令

打开 ir 的 debug 开关

echo 1 > /sys/class/remote/amremote/debug_enable

echo 8 > /proc/sys/kernel/printk

查看支持的 IR 协议

cat /sys/class/remote/amremote/protocol

```
console:/ # cat /sys/class/remote/amremote/protocol
protocol=NEC (0x1)
```

查看支持的遥控器用户码：

cat /sys/class/remote/amremote/map_tables

```
console:/ # cat /sys/class/remote/amremote/map_tables
0. 0xfb04,amlogic-remote-1
1. 0xfe01,amlogic-remote-2
2. 0xbd02,amlogic-remote-3
3. 0x7788,amlogic-remote-4
```

查看支持的遥控器物理键值

echo 0xfb04 > /sys/class/remote/amremote/keymap

cat /sys/class/remote/amremote/keymap

```
console:/ # echo 0xfb04 > /sys/class/remote/amremote/keymap
console:/ #
console:/ # cat /sys/class/remote/amremote/keymap
custom_code=0xfb04
custom_name=amlogic-remote-1
release_delay=80
map_size=51
fn_key_scancode = 0xffff
cursor_left_scancode = 0xffff
cursor_right_scancode = 0xffff
cursor_up_scancode = 0xffff
cursor_down_scancode = 0xffff
cursor_ok_scancode = 0xffff
keycode scancode
  67    0
  68    1
 136    2
 121    4
 122    5
 130    6
```

查看 IR linux 键值

getevent

```
console:/ # getevent
add device 1: /dev/input/event1
  name:     "cec_input"
could not get driver version for /dev/input/mice, Inappropriate ioctl for device
could not get driver version for /dev/input/mouse0, Inappropriate ioctl for device
add device 2: /dev/input/event5
  name:     "ir_keypad1"
add device 3: /dev/input/event2
  name:     "vad_keypad"
could not get driver version for /dev/input/mouse1, Inappropriate ioctl for device
add device 4: /dev/input/event3
  name:     "gpio_keypad"
add device 5: /dev/input/event4
  name:     "ir_keypad"
add device 6: /dev/input/event6
  name:     "adc_keypad"
add device 7: /dev/input/event0
  name:     "input_btrcu"
```

查看设备节点和加载的 kl 文件

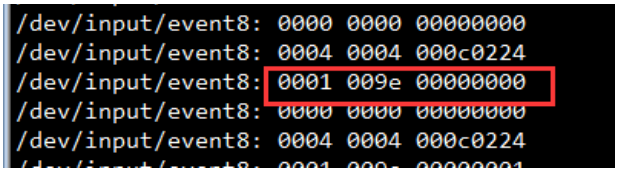
dumpsys input

```
2: ir_keypad1
  Classes: 0x00000029
  Path: /dev/input/event5
  Enabled: true
  Descriptor: 76d09a7d1f3b8d113bb56ed2568268f86d137837
  Location: keypad/input0
  ControllerNumber: 0
  UniqueId:
  Identifier: bus=0x0010, vendor=0x0002, product=0x0002, version=0x0200
  KeyLayoutFile: /vendor/usr/keylayout/Vendor_0002_Product_0002.kl
  KeyCharacterMapFile: /system/usr/keychars/Generic.kcm
  ConfigurationFile:
  HaveKeyboardLayoutOverlay: false
  VideoDevice: <none>
```

3.10.5 蓝牙遥控器配置

1. 先将蓝牙遥控器配对成功
2. 查看配对之后的遥控器对应的 input 设备，命令 dumpsys input

3. kernel 下输入 getevent，然后按遥控器某个键，例如返回键，出现如下打印



009e 则表示 linux 的按键值

4. 创建一个 kl 文件（可以复制系统中原有的）

名字为：**V**endor_[vendor]_Product_[product].kl

vendor 和 product 用步骤 2 中获取到的值，例如

Vendor_1d5a_Product_c086.kl

5. 修改 kl 文件内容如下，编辑对应的按键值。格式为

   key　linux 按键值　android 按键名

```
key 116   POWER
key 103   DPAD_UP
key 108   DPAD_DOWN
key 105   DPAD_LEFT
key 106   DPAD_RIGHT
key 353   DPAD_CENTER
key 158   BACK
key 217   ASSIST
key 172   HOME
key 115   VOLUME_UP
key 114   VOLUME_DOWN
```

6. 将 Vendor_1d5a_Product_c086.kl 放到/vendor/usr/keylayout/下，然后重启。使用 dumpsys input，查看对应的遥控器，则会发现使用了对应的键值配置文件：

```
    9: B17A Consumer Control
      Classes: 0x80000121
      Path: /dev/input/event8
      Enabled: true
      Descriptor: 89e4309f013270dbd4a91faf02832cc825440dd8
      Location:
      ControllerNumber: 0
      UniqueId: 0c:f3:01:99:5f:a6
      Identifier: bus=0x0005, vendor=0x1d5a, product=0xc086, version=0x0000
      KeyLayoutFile: /vendor/usr/keylayout/Vendor_1d5a_Product_c086.kl
      KeyCharacterMapFile: /system/usr/keychars/Generic.kcm
      ConfigurationFile:
      HaveKeyboardLayoutOverlay: false
      VideoDevice: <none>
```

如果发现此处使用的还是 Generic.kl，则有可能是 Vendor_1d5a_Product_c086.kl 文件中的格式不正确或者 android 键值错误导致。

配置好的 kl 文件放在 device/amlogic/common/products/mbox/

3.10.6 Android key layout 配置

Android Kl 文件是标准 linux 与 Android 的键值映射文件，如果没有为设备单独定义 kl 文件，系统会使用默认 Generic.kl。设备配置 kl 命名：
Vendor_[vendor]_Product_[product].kl

```
console:/ # ls /vendor/usr/keylayout/
Generic.kl                      Vendor_0957_Product_0006.kl
Vendor_0001_Product_0001.kl     Vendor_0c45_Product_1109.kl
Vendor_0002_Product_0002.kl     Vendor_1915_Product_0001.kl
Vendor_000d_Product_3838.kl     Vendor_1d5a_Product_c081.kl
Vendor_000d_Product_3839.kl     Vendor_1d5a_Product_c082.kl
Vendor_005d_Product_0001.kl     Vendor_7045_Product_1820.kl
Vendor_005d_Product_0002.kl     Vendor_7545_Product_0021.kl
Vendor_0484_Product_5738.kl     Vendor_7545_Product_0180.kl
Vendor_0508_Product_0110.kl     Vendor_7545_Product_0190.kl
```

Kl 文件左边对应 linux 中定义值，右边对应 android framwork 定义键值

```
key 1     ESCAPE
key 2     1
key 3     2
key 4     3
key 5     4
key 6     5
key 7     6
key 8     7
key 9     8
key 10    9
key 11    0
key 12    MINUS
key 13    EQUALS
key 14    DEL
key 15    TAB
```

ADC key 为示例配置：

1、通过 cat /proc/bus/input/devices 查看对应设备的 vendor 与 product 值

```
I: Bus=0010 Vendor=0001 Product=0001 Version=0100
N: Name="adc_keypad"
P: Phys=adc_keypad/input0
S: Sysfs=/devices/platform/adc_keypad/input/input6
U: Uniq=
H: Handlers=event6
B: PROP=0
B: EV=100003
B: KEY=1c0000 0 0 0
```

Adc_keypad 使用的 kl 为 Vendor_0001_Product_0001.kl

3、修改 Vendor_0001_Product_0001.kl 文件内容，编辑对应的按键值；

格式：key　linux 键值　Android 键值

```
key 115     VOLUME_UP
key 114     VOLUME_DOWN
key 113     VOLUME_MUTE
```

/frameworks/native/include/input/InputEventLabels.h

InputEventLabel KEYCODES[]数组中 Android 键值定义

3.11 Audio 配置

嵌入式设备的音频系统可以被划分为板载硬件（Machine）、Soc（Platform）、Codec 三大部分，方便软件的抽象和重用；我们一般需要做的是就是配置 platform 里面 tdm ,cpu_dai 和 codec_dai 连接起来，配置 ch、sampleRate、位深、clk 等。以 S905Y4 内置 codec 配置示例：
common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222_drm.dts

3.11.1 Machine 配置

```
auge_sound {
    compatible = "amlogic, auge-sound-card";
    aml-audio-card,name = "AML-AUGESOUND";

    /*avout mute gpio*/
    avout_mute-gpios = <&gpio GPIOH_5 GPIO_ACTIVE_HIGH>;
    spk_mute-gpios = <&gpio GPIOH_8 GPIO_ACTIVE_LOW>;
```

3.11.2 dai-link 配置

```
aml-audio-card,dai-link@2 {
    format = "i2s";
    mclk-fs = <256>;
    //continuous-clock;
    //bitclock-inversion;
    //frame-inversion;
    /* master mode */
    bitclock-master = <&tdmc>;
    frame-master = <&tdmc>;
    /* slave mode */
    //bitclock-master = <&tdmccodec>;
    //frame-master = <&tdmccodec>;
    /* suffix-name, sync with android audio hal used for */
    suffix-name = "alsaPORT-i2s";
    cpu {
        sound-dai = <&tdmc>;
        dai-tdm-slot-tx-mask = <1 1>;
        dai-tdm-slot-rx-mask = <1 1>;
        dai-tdm-slot-num = <2>;
        dai-tdm-slot-width = <32>;
        system-clock-frequency = <12288000>;
    };
    tdmccodec: codec {
        sound-dai = <&amlogic_codec /*&ad82584f_62*/>;
    };
};
```


dai-link@2 ： 第二个 dai

format = "i2s" : cpu dai 与 codec dai 通信方式 I2S， 支持 i2s/dsp_a/dsp_b

mclk-fs = <256>; ：主时钟频率配置，I2S 一般是 256*48K=12.288M

//continuous-clock; ：时钟是否持续输出

//bitclock-inversion; ： 非标准 I2S，用于配置 I2S bit 时钟反转

//frame-inversion; ： 非标准 I2S，用于 LR 时钟反转

bitclock-master = <&tdmc>; ：soc I2S 主时钟，bit clock 输出

frame-master = <&tdmc>; ：soc I2S 主时钟，LR clock 输出

//bitclock-master = <&tdmccodec>; ： soc I2S 从时钟，codec 输出 bit clock

//frame-master = <&tdmccodec>; :soc I2S 从时钟，codec 输出 LR clock

sound-dai = <&tdmc>; : cpu dai 选择配置

dai-tdm-slot-tx-mask = <1 1>; 输出 2ch,即左右声道；如果是 8ch 则为 8 个 1

dai-tdm-slot-rx-mask = <1 1>; 输入 2ch,即左右声道；如果是 8ch 则为 8 个 1

dai-tdm-slot-num = <2>; 配置两个声道

dai-tdm-slot-width = <32>;配置位宽 32bit

sound-dai = <&amlogic_codec /*&ad82584f_62*/>; : 功放选择配置，使用内置 DSP

3.11.3 I2S/TDM 配置

```
tdmc: tdm@2 {
    compatible = "amlogic, t5-snd-tdmc";
    #sound-dai-cells = <0>;
    dai-tdm-lane-slot-mask-in = <1>;
    dai-tdm-lane-slot-mask-out = <1>;
    dai-tdm-clk-sel = <2>;
    clocks = <&clkaudio CLKID_AUDIO_MCLK_C
            &clkaudio CLKID_AUDIO_MCLK_PAD0
            &clkc CLKID_MPLL2>;
    clock-names = "mclk", "mclk_pad", "clk_srcpll";
    /*
     * 0: tdmout_a;
     * 1: tdmout_b;
     * 2: tdmout_c;
     * 3: spdifout;
     * 4: spdifout_b;
     */
    samesource_sel = <3>;

    /*enable default mclk(12.288M), before extern codec start*/
    start_clk_enable = <1>;

    /*tdm clk tuning enable*/
    clk_tuning_enable = <1>;

    status = "okay";
};
```

dai-tdm-lane-slot-mask-in = <1>; ： 输入一个 lane，即 2ch

dai-tdm-lane-slot-mask-out = <1>; ： 输出一个 lane，即 2ch

samesource_sel = <3>; ：tdmc out 输出同时，将 tdmc 音频数据 copy 到 spdifout 上

start_clk_enable = <1>; ：上电后输出主时钟 MCLK 12.288M，使能后不需要播放时才输出

clk_tuning_enable = <1>; 动态调 clk

3.11.4 I2S PINMUX 配置

以下是 S905Y4 tdma_I2S 引脚 pinmux 配置示例

```
&pinctrl_audio {
    tdm_d0_pins: tdm_d0_pin {
        mux {
            groups = "tdm_d0";
            function = "tdmouta_lane0";
        };
    };

    tdm_d1_pins: tdm_d1_pin {
        mux {
            groups = "tdm_d1";
            function = "tdmina_lane0";
        };
    };

    tdma_clk_pins: tdm_clk_pin {
        mux {
            groups = "tdm_sclk0", "tdm_lrclk0";
            function = "tdm_clk_outa";
        };
    };
};
```

3.11.5 codec 配置

以下是 S905Y4 内置 codec 驱动配置

```
amlogic_codec:t9015{
    #sound-dai-cells = <0>;
    compatible = "amlogic, s4_codec_T9015";
    reg = <0x0 0xFE01A000 0x0 0x2000>;
    tocodec_inout = <2>;
    tdmout_index = <2>;
    ch0_sel = <0>;
    ch1_sel = <1>;

    reset-names = "acodec";
    resets = <&reset RESET_ACODEC>;

    status = "okay";
};
```

3.11.6 Audio 调试命令

查看当前所有声卡

cat /proc/asound/pcm

```
console:/ $ cat /proc/asound/pcm
00-00: TDM-A-dummy-alsaPORT-pcm soc:dummy-0 :  : playback 1 : capture 1
00-01: TDM-B-dummy-alsaPORT-i2s2hdmi soc:dummy-1 :  : playback 1 : capture 1
00-02: TDM-C-T9015-audio-hifi-alsaPORT-i2s fe01a000.t9015-2 :  : playback 1 : capture 1
00-03: PDM-dummy-alsaPORT-pdm-builtinmic soc:dummy-3 :  : capture 1
00-04: SPDIF-dummy-alsaPORT-spdif soc:dummy-4 :  : playback 1 : capture 1
00-05: SPDIF-B-dummy-alsaPORT-spdifb soc:dummy-5 :  : playback 1
00-06: LOOPBACK-A-dummy-alsaPORT-loopback soc:dummy-6 :  : capture 1
```

查看当前所有声卡状态

cat /proc/asound/card0/pcm*p/sub0/status

```
at /proc/asound/card0/pcm2p/sub0/status
state: RUNNING
owner_pid   : 495
trigger_time: 10.632063505
tstamp      : 3206.172866861
delay       : 2944
avail       : 128
avail_max   : 0
-----
hw_ptr      : 153383168
appl_ptr    : 153386240
```

查看当前声卡设置参数

cat /proc/asound/card0/pcm*p/sub0/hw_params

```
at /proc/asound/card0/pcm2p/sub0/hw_params
access: RW_INTERLEAVED
format: S16_LE
subformat: STD
channels: 2
rate: 48000 (48000/1)
period_size: 512
buffer_size: 3072
```

Tinyalsa 工具使用命令

Tinyplay 播放 wav 格式音频

tinyplay 16-768.wav -c 4 -p 10000

Usage: tinyplay file.wav [-D card] [-d device] [-p period_size] [-n n_periods]

tinycap 录音 wav 格式音频

Usage: tinycap file.wav [-D card] [-d device] [-c channels] [-r rate] [-b bits] [-p period_size] [-n n_periods] [-T capture time]

tinymix 设置 ALSA 属性

3.12 Ethernet 配置

打开驱动配置：common/arch/arm64/configs/meson64_a64_R_defconfig

CONFIG_STMMAC_ETH=m

CONFIG_DWMAC_MESON=m

3.12.1 内置 phy 配置

S905y4 公版内置百兆网卡，默认使能内置 phy，100M，RMII。

common\arch\arm64\boot\dts\amlogic\s4_s905y4_ap222_drm.dts

```
&ethmac {
    status = "okay";
    phy-handle = <&internal_ephy>;
    phy-mode = "rmii";
};
```

status = "okay"; //开启或关闭内置 phy

common\arch\arm64\boot\dts\amlogic\meson-s4.dtsi

```
ethmac: ethernet@fdc00000 {
    compatible = "amlogic,meson-axg-dwmac",
                 "snps,dwmac-3.70a",
                 "snps,dwmac";
    reg = <0x0 0xfdc00000 0x0 0x10000>,
          <0x0 0xfe024000 0x0 0x8>;
    interrupts = <GIC_SPI 74 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "macirq";
    power-domains = <&pwrdm PDID_S4_ETH>;
    clocks = <&clkc CLKID_ETH>,
         <&clkc CLKID_FCLK_DIV2>,
         <&clkc CLKID_MPLL2>;
    clock-names = "stmmaceth", "clkin0", "clkin1";
    rx-fifo-depth = <4096>;
    tx-fifo-depth = <2048>;
    status = "disabled";

    mdio0: mdio {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "snps,dwmac-mdio";
    };
};
```

```
eth_phy: mdio-multiplexer@28000 {
    compatible = "amlogic,g12a-mdio-mux";
    reg = <0x0 0x28000 0x0 0xa4>;

    clocks = <&clkc CLKID_ETHPHY>,
         <&xtal>,
         <&clkc CLKID_MPLL_50M>;
    clock-names = "pclk", "clkin0", "clkin1";
    mdio-parent-bus = <&mdio0>;
    #address-cells = <1>;
    #size-cells = <0>;
    enet_type = <5>;
    tx_amp_src = <0xFE010330>;

    ext_mdio: mdio@0 {
        reg = <0>;
        #address-cells = <1>;
        #size-cells = <0>;
    };

    int_mdio: mdio@1 {
        reg = <1>;
        #address-cells = <1>;
        #size-cells = <0>;

        internal_ephy: ethernet_phy@8 {
            compatible = "ethernet-phy-id0180.3301",
                         "ethernet-phy-ieee802.3-c22";
            interrupts = <GIC_SPI 75 IRQ_TYPE_LEVEL_HIGH>;
            reg = <8>;
            max-speed = <100>;
        };
    };
};
```


3.12.2 调试命令

查看 eth 状态以及 IP 地址

ifconfig eth0

```
console:/ # ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:ad:37:01:d9:bb  Driver meson8b-dwmac
          inet addr:192.168.4.243  Bcast:192.168.4.255  Mask:255.255.255.0
          inet6 addr: fe80::89de:312a:dc31:6f3c/64 Scope: Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:18 errors:0 dropped:0 overruns:0 frame:0
          TX packets:41 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2413 TX bytes:6008
          Interrupt:44
```

查看 link 状态

cat /sys/class/ethernet/linkspeed

```
console:/ # cat /sys/class/ethernet/linkspeed
[ 8086.694494@3]  meson8b-dwmac fdc00000.ethernet eth0: Link is Up - 100Mbps/Full - flow control rx/tx
link status: link
```

3.13 UART 配置

3.13.1 Kernel UART 配置

S905Y4 UARTA 配置示例：common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
uart_A: serial@fe078000 {
        compatible = "amlogic,meson-uart";
        reg = <0x0 0xfe078000 0x0 0x18>;
        interrupts = <0 168 1>;
        status = "disabled";
        clocks = <&xtal
                  &clkc CLKID_UART◆A>;
        clock-names = "clk_uart",
            "clk_gate";
        xtal_tick_en = <3>;
        fifosize = < 64 >;
        pinctrl-names = "default";
        pinctrl-0 = <&a_uart_pins>;
};
```

```
a_uart_pins:a_uart {
    mux {
        groups = "uart_a_tx",
                 "uart_a_rx",
                 "uart_a_cts",
                 "uart_a_rts";
        function = "uart_a";
    };
};
```

3.13.2 系统 UART 波特率配置

S905Y4 示例：/bootloader/uboot-repo/bl33/v2019/board/amlogic/configs/s4_ap201.h

```
--- a/board/amlogic/configs/s4_ap201.h
+++ b/board/amlogic/configs/s4_ap201.h
@@ -29,7 +29,7 @@
 #endif

 /*low console baudrate*/
-#define CONFIG_LOW_CONSOLE_BAUD                         0
+#define CONFIG_LOW_CONSOLE_BAUD                         1

 /* Enable ir remote wake up for bl30 */
 #define AML_IR_REMOTE_POWER_UP_KEY_VAL1 0xef10fe01 //amlogic tv ir --- power
@@ -110,7 +110,7 @@
         "fatload_dev=usb\0"\
         "fs_type=""rootfstype=ramfs""\0"\
         "initargs="\
-            "init=/init console=ttyS0,921600 no_console_suspend earlycon=aml-uart,0xfe07a000 "\
+            "init=/init console=ttyS0,115200 no_console_suspend earlycon=aml-uart,0xfe07a000 "\
             "ramoops.pstore_en=1 ramoops.record_size=0x8000 ramoops.console_size=0x4000 loop.max_part=4 "\
             "\0"\
         "upgrade_check="\
```

3.13.3 UART 调试命令

UART 波特率支持设置 9600/115200/921600/2M/3M/4M

stty -F /dev/ttyS0 115200

UART 发送

```
console:/ # echo "Hello World" > /dev/ttyS0
Hello World
```

3.14 Watchdog 配置

Watchdog 主要用于监测系统运行情况，一旦出现卡死、死锁、死机等情况，就会重启系统

3.14.1 Kernel watchdog 配置

示例：common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
watchdog@2100 {
    compatible = "amlogic,meson-sc2-wdt";
    status = "okay";
    /* 0:userspace, 1:kernel */
    amlogic,feed_watchdog_mode = <1>;
    reg = <0x0 0x2100 0x0 0x10>;
    clocks = <&xtal>;
};
```

amlogic,feed_watchdog_mode = <1>; //喂狗方式，0：android 守护进程操作节点喂狗 1：内核线程喂狗

3.14.2 Android watchdog 应用喂狗

DTS 设置 amlogic,feed_watchdog_mode = <0>;并自启动 watchdog 服务，S905Y4 示例：device/amlogic/common/products/mbox/init.amlogic.system.rc

```
--- a/products/mbox/init.amlogic.system.rc
+++ b/products/mbox/init.amlogic.system.rc
@@ -79,10 +79,10 @@ service fuse_loop /system/bin/sdcard -u 1023 -g 1023 /mnt/media_rw/loop /storage
     disabled

 # Set watchdog timer to 30 seconds and pet it every 10 seconds to get a 20 second margin
-#service watchdogd /sbin/watchdogd 10 20
-#    class core
-#    disabled
-#    seclabel u:r:watchdogd:s0
+service watchdogd /sbin/watchdogd 10 20
+    class core
+    disabled
+    seclabel u:r:watchdogd:s0
```

Watchdog code : system/core/watchdogd/watchdogd.cpp 通过对/dev/watchdog 节点进行写操作喂狗。

3.15 CVBS 配置

CVBSOUT 一般不允许项目自己另外设计修改，S905Y4 示例配置：

```
cvbsout {
    compatible = "amlogic, cvbsout-s4";
    status = "okay";

    /* clk path */
    /* 0:vid_pll vid2_clk */
    /* 1:gp0_pll vid2_clk */
    /* 2:vid_pll vid1_clk */
    /* 3:gp0_pll vid1_clk */
    clk_path = <0>;

    /* performance: reg_address, reg_value */
    /* s4 */
    performance_pal = <0x1bf0  0x9 /* default Matrx625 CTCC value*/
        0x1b12  0x80c0
        0x1b05  0xf7
        0x1b56  0x333
        0x1c59  0xf25c
        0xffff  0x0>; /* ending flag */
    performance = <0x1bf0  0x9 /* ccitt033 SVA value */
        0x1b12  0x8040
        0x1b05  0xfd
        0x1b56  0x333
        0x1c59  0xf654
        0xffff  0x0>; /* ending flag */
    performance_ntsc = <0x1bf0  0x9
        0x1b12  0x8020
        0x1b03  0x0
        0x1b04  0x0
        0x1b05  0x0
        0x1b06  0x0
        0x1b56  0x333
        0x1c59  0xf850
        0xffff  0x0>; /* ending flag */
};
```

3.16 HDMI 配置

3.16.1 HDMI TX 配置

HDMI TX 一般不允许项目自己另外设计修改，S905Y4 示例：

common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
amhdmitx: amhdmitx{
    compatible = "amlogic, amhdmitx-sc2";
    dev_name = "amhdmitx";
    status = "disabled";
    vend-data = <&vend_data>;
    pinctrl-names="default";
    pinctrl-0=<&hdmitx_hpd &hdmitx_ddc>;
    clock-names = "venci_top_gate",
        "venci_0_gate",
        "venci_1_gate",
        "hdmi_vapb_clk",
        "hdmi_vpu_clk";
    /* refer to sc2-system-registers.docx */
    interrupts = <0 204 1
        0 197 1>;
    interrupt-names = "hdmitx_hpd", "viu1_vsync";
    /* refer to hdmi_tx_module.h */
    ic_type = <15>;
    hdmi_rext = <1300>; /* HDMI Rext resistor uses 1.3k ohm, not 1.5k */
    reg =   <0x0 0x00000000 0x0 0x000>, /* reserved */
        <0x0 0xff000000 0x0 0x40000>,
        <0x0 0x00000000 0x0 0x000>, /* reserved */
        <0x0 0xfe308000 0x0 0x8000>,
        <0x0 0xfe300000 0x0 0x8000>,
        <0x0 0xfe032000 0x0 0x100>,
        <0x0 0xfe008000 0x0 0x400>,
        <0x0 0xfe00c000 0x0 0x800>,
        <0x0 0xfe002000 0x0 0x400>,
        <0x0 0xfe010000 0x0 0x100>,
        <0x0 0xfe000000 0x0 0x2000>;
```

3.16.2 HDMI CEC 配置

device/amlogic/oppen/vendor_prop.mk 盒子设置为 4，TV 设置为 0

```
PRODUCT_PROPERTY_OVERRIDES += \
    ro.hdmi.device_type=4 \
```

Kernel dts 配置

common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
aocec: aocec {
    compatible = "amlogic, aocec-s4";
    dev_name = "aocec";
    status = "okay";
    vendor_name = "Amlogic"; /* Max Chars: 8     */
    /* Refer to the following URL at:
     * http://standards.ieee.org/develop/regauth/oui/oui.txt
     */
    vendor_id = <0xffffff>;
    product_desc = "S4"; /* Max Chars: 16    */
    cec_osd_string = "AML_MBOX"; /* Max Chars: 14    */
    cec_version = <5>;/*5:1.4;6:2.0*/
    port_num = <1>;
    output = <1>;
    cec_sel = <1>;/*1:use one ip, 2:use 2 ip*/
    ee_cec;
    arc_port_mask = <0x1>;
    interrupts = <GIC_SPI 180 IRQ_TYPE_EDGE_RISING/*0:snps*/
        GIC_SPI 179 IRQ_TYPE_EDGE_RISING>;/*1:ts*/
    interrupt-names = "hdmi_aocecb","hdmi_aocec";
    pinctrl-names = "default","hdmitx_aocecb","cec_pin_sleep";
    pinctrl-0=<&eecec_a>;
    pinctrl-1=<&eecec_b>;
    pinctrl-2=<&eecec_b>;
    clocks = <&clkc CLKID_CECA_32K_CLKOUT>,
        <&clkc CLKID_CECB_32K_CLKOUT>;
    clock-names = "ceca_clk","cecb_clk";
    reg = <0x0 0xfe044000 0x0 0x2ff
           0x0 0xfe010000 0x0 0xfff
```

3.16.3 HDMI 调试命令

uboot 下 HDMI 输出测试

hdmitx output bist 1

HDMI control 信息查询

dumpsys hdmi_control

```
130|console:/ # dumpsys hdmi_control
mProhibitMode: false
mPowerStatus: 0
System_settings:
  mHdmiControlEnabled: true
  mMhlInputChangeEnabled: true
  mSystemAudioActivated: false
  mHdmiCecVolumeControlEnabled true
mMhlController:
mPortInfo:
  port_id: 0, type: HDMI_OUT, address: 0x1000, cec: true, arc: true, mhl: false
mCecController:
  HdmiCecLocalDevice #4:
    mDeviceType: 4
    mAddress: 4
    mPreferredAddress: 4
    mDeviceInfo: CEC: logical_address: 0x04 device_type: 4 vendor_id: 0xFFFFFF display_name: ap201 power_status: 0 physical_address: 0x1000 port_id: 0
    mActiveSource: (0x04, 0x1000)
    mActiveRoutingPath: 0x0000
    mRoutingControlFeatureEnabled: true
    mIsActiveSource: true
    mAutoTvOff:true
    mOneTouchPlayEnabed:true
    mAutoLanguageChange:false
  CEC message history:
    [s] time=2022-05-09 05:50:46 message=<Give Device Power Status> 40:8F
    [s] time=2022-05-09 05:50:46 message=<Report Physical Address> 4F:84:10:00:04
    [s] time=2022-05-09 05:50:46 message=<Device Vendor Id> 4F:87:FF:FF:FF
    [s] time=2022-05-09 05:50:46 message=<Give Device Vendor Id> 40:8C
    [s] time=2022-05-09 05:50:46 message=<Set Osd Name> 40:47:61:70:32:30:31
    [s] time=2022-05-09 05:50:46 message=<Give System Audio Mode Status> 45:7D
    [s] time=2022-05-09 05:50:46 message=<Text View On> 40:0D
    [s] time=2022-05-09 05:50:46 message=<Active Source> 4F:82:10:00
```

HDMI 连接状态查询

cat/sys/class/amhdmitx/amhdmitx0/hpd_state //0 未连接 1 已连接

查询电视支持显示模式

cat /sys/class/amhdmitx/amhdmitx0/disp_cap

```
at /sys/class/amhdmitx/amhdmitx0/disp_cap
480i60hz
576i50hz
480p60hz
576p50hz
720p60hz
1080i60hz
1080p60hz*
720p50hz
1080i50hz
1080p50hz
640x480p60hz
800x600p60hz
1024x768p60hz
1152x864p75hz
1280x960p60hz
1280x1024p60hz
1360x768p60hz
1440x900p60hz
1680x1050p60hz
```

查询电视是否支持某格式

echo 2160p60hz444 12bit > /sys/class/amhdmitx/amhdmitx0/valid_mode

cat /sys/class/amhdmitx/amhdmitx0/valid_mode //0 不支持 1 支持

```
console:/ # echo 2160p60hz444 12bit > /sys/class/amhdmitx/amhdmitx0/valid_mode
at /sys/class/amhdmitx/amhdmitx0/valid_mode
0
```

查询 HDMI 输出格式与色彩位数

cat /sys/class/amhdmitx/amhdmitx0/attr

```
console:/ # cat /sys/class/amhdmitx/amhdmitx0/attr
422,12bit
```

查询 HDMI 输出分辨率

cat sys/class/display/mode

```
console:/ # cat sys/class/display/mode
1080p60hz
```

Bist 模式验证命令，观察颜色是否正常

echo bist2100 > /sys/class/amhdmitx/amhdmitx0/debug

设置分辨率

echo 1 > /sys/class/display/debug

echo 1080p60hz > /sys/class/display/mode

查询电视支持 HDCP 版本

cat /sys/class/amhdmitx/amhdmitx0/hdcp_ver

```
console:/ # cat /sys/class/amhdmitx/amhdmitx0/hdcp_ver
14
```

查看当前 HDCP 模式

cat /sys/class/amhdmitx/amhdmitx0/hdcp_mode

设置当前 HDCP 模式

echo 1 > /sys/class/amhdmitx/amhdmitx0/hdcp_mode //1: hdcp14 2:hdcp22

查看烧录 HDCP provision key

cat /sys/class/amhdmitx/amhdmitx0/hdcp_lstore

AV mute 切换

echo -1 > /sys/devices/virtual/amhdmitx/amhdmitx0/avmute //1：准备切换 -1：切换完成

3.17 Thermal 配置

CPU 的发热与当前运行的频率、核数等有关系。CPU 热管理实际就是通过一些途径获取 ip 的温度，当超过用户配置的阈值后，通过限制 CPU 的最大核数、频率等手段，来迫使 CPU 的温度降下来。平台的默认温控都是开启的。根据项目需要调整温控参数，及关闭温控。

3.17.1 Uboot Thermal 配置

启动进到 bootloader 后，如果获取到的当前温度比较定义的温度（一般设置 90℃）高，则会停在 bootloader 做 sleep 冷却并且轮询，直到温度低于定义的温度后才会继续执行,如果关闭，可能出现的风险，启动时温度很高，进 kernel 之后 CPU loading 更大，温度继续上升，很快又触发了 shutdown 或者 reboot，如果是 reboot,则会进入高温重启的死循环。

如果需要关闭温控，只需要注释下列宏。当然也可以根据项目要求调整温度阀值。

文件路径：

bootloader/uboot-repo/bl33/v2019/arch/arm/include/asm/arch-s4/tsensor.h

3.17.2 Kernel Thermal 配置

通过 DTS 进行配置，分为两部分：thermal sensor 配置和 Thermal 参数配置

S905Y4 温控示例: common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
p_tsensor: p_tsensor@fe020000 {
    compatible = "amlogic, r1p1-tsensor";
    status = "okay";
    reg = <0x0 0xfe020000 0x0 0x50>;
    tsensor_id = <1>;
    cal_type = <0x11>;
    cal_coeff = <324 424 3159 9411>;
    rtemp = <110000>;
    interrupts = <GIC_SPI 30 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&clkc CLKID_TS_CLK_GATE>;
    clock-names = "ts_comp";
    #thermal-sensor-cells = <1>;
};
```

rtemp = <110000>; //硬件重启温度

Thermal 参数配置：

```
thermal-zones {
    soc_thermal: soc_thermal {
        polling-delay = <1000>;
        polling-delay-passive = <100>;
        sustainable-power = <1160>;
        thermal-sensors = <&p_tsensor 0>;
        trips {
            pswitch_on: trip-point@0 {
                temperature = <85000>;
                hysteresis = <5000>;
                type = "passive";
            };
            pcontrol: trip-point@1 {
                temperature = <95000>;
                hysteresis = <5000>;
                type = "passive";
            };
            pcritical: trip-point@2 {
                temperature = <105000>;
                hysteresis = <1000>;
                type = "critical";
            };
        };

        cooling-maps {
            cpufreq_cooling_map {
                trip = <&pcontrol>;
                cooling-device = <&CPU0 0 8>;
                contribution = <1024>;
            };
            gpufreq_cooling_map {
                trip = <&pcontrol>;
                cooling-device = <&gpu 0 3>;
                contribution = <1024>;
            };
        };
    };
};/*thermal zone end*/
```


polling-delay = <1000>; //温度小于阀值时，轮询时间间隔 1000ms

polling-delay-passive = <100>; //温度大于阀值时，轮询时间间隔 100ms

sustainable-power = <1160>; //可持续功率 1160mw

thermal-sensors = <&p_tsensor 0>; //选择使用的 sensor

pswitch_on: trip-point@0 {

temperature = <85000>; //switch on 开启温度为 85 摄氏度,IPA 温控开启

```
        hysteresis = <5000>;        //switch on  关闭温度为 85-5=80 摄氏度
        type = "passive";           //冷却类型，被动
 };
pcontrol: trip-point@1 {
        temperature = <95000>;  //在 95 摄氏度开启降频
        hysteresis = <5000>;        //95-5=90  摄氏度关闭温控功能
        type = "passive";
};
pcritical: trip-point@2 {
        temperature = <105000>;  //在 105 摄氏度关机
        hysteresis = <1000>;
        type = "critical";
};
```

3.17.3 温度调试命令

Uboot 读取温度

read_temp;

```
s4_ap222# read_temp
read temp
type: ret = 840000e8
pll tsensor avg: 0x1ff0, u_efuse: 0xe8
temp1: 41
read the thermal
s4_ap222#
```

Kernel 读取温度

cat /sys/class/thermal/thermal_zone0/temp

```
console:/sys/class/thermal/thermal_zone0 # cat temp
56900
```

关闭温控

echo disabled > /sys/class/thermal/thermal_zone0/mode

查看温控触发点

cat /sys/class/thermal/thermal_zone0/trip_point_1_type

cat /sys/class/thermal/thermal_zone0/trip_point_1_temp

cat /sys/class/thermal/thermal_zone0/trip_point_2_type

cat /sys/class/thermal/thermal_zone0/trip_point_2_temp

...

修改温控触发点

echo 90000 > /sys/class/thermal/thermal_zone0/trip_point_1_temp

3.18 VDDCPU & VDDEE 配置

3.18.1 VDDCPU PDVFS

这项技术根据芯片运行的应用程序的计算需求，动态调整电压和频率，在不需要高性能时，降低电压和频率，以降低功耗；在需要高性能时，提高电压和频率，以提高性能，从而达到兼顾性能而又节能的目的。

从 Android R/S 开始 PVDFS 架构设计了 4 份 CPU 频率&电压 table0-table3,系统启动阶段会自动根据硬件 load 不同的 table,在做硬件稳定性测试的之前需要知道当前板子 load 的是那份 table,通过开机 log check tables_index

		频率	推荐电压（V），先查看开机 log 信息，找到对应的 tables 信息再对应电压：meson_cpufreq: choose_cpufreq_tables_index:tables_index ？			测试结果对应任何一个表格都 OK；	
			tables_index 0/1	tables_index 2	tables_index 3	测试结果（V）	结果判定
	VDDCPU 动态调压	100MHz	0.769	0.759	0.759		Pass?
		250MHz	0.769	0.759	0.759		Pass?
		500MHz	0.769	0.759	0.759		Pass?
		666MHz	0.769	0.759	0.759		Pass?

以 S905Y4 公版示例 VDDCPU 电压 PDVFS 配置：
common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
s905y4_opp_table0: s905y4_opp_table0 {
    compatible = "operating-points-v2";
    status = "okay";
    opp-shared;

    opp00 {
        opp-hz = /bits/ 64 <100000000>;
        opp-microvolt = <769000>;
    };
    opp01 {
        opp-hz = /bits/ 64 <250000000>;
        opp-microvolt = <769000>;
    };
    opp02 {
        opp-hz = /bits/ 64 <500000000>;
        opp-microvolt = <769000>;
    };
    opp03 {
        opp-hz = /bits/ 64 <666000000>;
        opp-microvolt = <769000>;
    };
    opp04 {
        opp-hz = /bits/ 64 <1000000000>;
        opp-microvolt = <769000>;
    };
    opp05 {
        opp-hz = /bits/ 64 <1200000000>;
        opp-microvolt = <769000>;
    };
    opp06 {
        opp-hz = /bits/ 64 <1404000000>;
        opp-microvolt = <799000>;
```

s905y4_opp_table0 //cpu0 opp table

opp-hz = /bits/ 64 <1200000000>; //动态调压的频率 1.2G HZ

opp-microvolt = <769000>; //动态调压的电压 0.769 v

3.18.2 VDDEE 配置

以 S905Y4 示例：bootloader/uboot-repo/bl33/v2019/board/amlogic/configs/s4_ap222.h

```
#define AML_VCCK_INIT_VOLTAGE     799       //VCCK（VDDCPU）power up voltage

#define AML_VDDEE_INIT_VOLTAGE    800       // VDDEE power up voltage
```

3.18.3 CPU DVFS 调试命令

查看所有支持的频率列表

cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies

```
ystem/cpu/cpu0/cpufreq/scaling_available_frequencies                    <
100000 250000 500000 666000 1000000 1200000 1404000 1500000 1608000 1704000 1800000 1908000 2004000
```

查看当前 cpu 频点

cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq

```
/sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq                   <
2004000
```

硬件稳定性检查表中有会相关 VDDCPU 动态电压评估测试项，设置定频测试 VDDCPU 电压方法：

1. 设置 performance 模式

echo performance > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor

2. 设置 CPU 工作频率

echo 1500000 > /sys/devices/system/cpu/cpu0/cpufreq/scaling_max_freq

echo 1500000 > /sys/devices/system/cpu/cpu0/cpufreq/scaling_min_freq

4. 系统相关

4.1 Unifykey 配置

Amlogic Unifykey 是一个用于统一管理 Amlogic 全平台 key 的子系统， 为用户提供了简单的差异化配置方法以及充分的 key 管理方法， 且用户也不需要关心平台底层存储介质的差异。 特别在 unifykey 在实现过程中，提供充分的 key 管理功能， 也特别考虑了 key 的安全属性。

4.1.1 Kernel unifykey 配置

S905Y4 securitykey 定义：common/arch/arm64/boot/dts/amlogic/meson-s4.dtsi

```
securitykey {
    compatible = "aml, securitykey";
    storage_query = <0x82000060>;
    storage_read = <0x82000061>;
    storage_write = <0x82000062>;
    storage_tell = <0x82000063>;
    storage_verify = <0x82000064>;
    storage_status = <0x82000065>;
    storage_list = <0x82000067>;
    storage_remove = <0x82000068>;
    storage_in_func = <0x82000023>;
    storage_out_func = <0x82000024>;
    storage_block_func = <0x82000025>;
    storage_size_func = <0x82000027>;
    storage_set_enctype = <0x8200006A>;
    storage_get_enctype = <0x8200006B>;
    storage_version = <0x8200006C>;
};
```

securitykey 定义了 bl31 里所调用函数的 function id，一般这里不需要修改。

S905Y4 Unifykey: common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222.dts

```
unifykey{
    compatible = "amlogic,unifykey";
    status = "ok";
    unifykey-num = <19>;
    unifykey-index-0 = <&keysn_0>;
    unifykey-index-1 = <&keysn_1>;
    unifykey-index-2 = <&keysn_2>;
    unifykey-index-3 = <&keysn_3>;
    unifykey-index-4 = <&keysn_4>;
    unifykey-index-5 = <&keysn_5>;
    unifykey-index-6 = <&keysn_6>;
    unifykey-index-7 = <&keysn_7>;
    unifykey-index-8 = <&keysn_8>;
    unifykey-index-9 = <&keysn_9>;
    unifykey-index-10= <&keysn_10>;
    unifykey-index-11= <&keysn_11>;
    unifykey-index-12= <&keysn_12>;
    unifykey-index-13= <&keysn_13>;
    unifykey-index-14= <&keysn_14>;
    unifykey-index-15= <&keysn_15>;
    unifykey-index-16= <&keysn_16>;
    unifykey-index-17= <&keysn_17>;
    unifykey-index-18= <&keysn_18>;

    keysn_0: key_0{
        key-name = "usid";
        key-device = "normal";
        key-permit = "read","write","del";
    };
```

unifykey: 定义了平台支持的所有 key 及对应属性。

key-device = "normal";

Noraml (存储于 nand/emmc, 可获取 key 内容)

Secure (存储于 nand/emmc, 可获取 key 内容的 hash 值)

Efuse (存储于 efuse)

说明：Noraml 与 secure unifykey 存储于 emmc/nand 的 rsv 分区中，这些 key 不在逻辑分区中，所以一般情况是不会被用户擦除的。

key-type = "mac";

mac (使用冒号分隔的字符,02:ad:37:01:d9:bb)

sha1（以 20 字节的 sha1 值）

dhcp（dhcp2 专用的格式）

raw （可以直接烧录，默认方式）

4.1.2 unifykey 调试命令

Uboot 读取 unifykey 使用说明，S905Y4 示例：

bootloader/uboot-repo/bl33/v2019/board/amlogic/configs/s4_ap201.h

```
"if keyman init 0x1234; then "\
    "if keyman read usid ${loadaddr} str; then "\
        "setenv bootargs ${bootargs} androidboot.serialno=${usid};"\
        "setenv serial ${usid}; setenv serial# ${usid};"\
```

keyman init 0x1234 //初始化 unifykey

keyman read usid ${loadaddr} str //字符方式读取 unifykey 值

Kernel Unifykey 节点使用说明

```
console:/ # cat /sys/class/unifykeys/help
Usage:
echo 1 > attach //initialise unifykeys
cat lock //get lock status
//if lock=1,you must wait, lock=0, you can go on
//so you must set unifykey lock first, then do other operations
echo 1 > lock //1:locked, 0:unlocked
echo "key name" > name //set current key name->"key name"
cat name //get current key name
echo "key value" > write //set current key value->"key value"
cat read //get current key value
cat size //get current key value
cat exist //get whether current key is exist or not
cat list //get all unifykeys
cat version //get unifykeys versions
//at last, you must set lock=0 when you has done all operations
echo 0 > lock //set unlock
```

查看所有 unifykey 信息

```
console:/ # cat /sys/class/unifykeys/list
19 keys installed
00: usid, normal, 3
01: mac, normal, 3
02: hdcp, secure, 3
03: secure_boot_set, efuse, 2
04: mac_bt, normal, 3
05: mac_wifi, normal, 3
06: hdcp2_tx, normal, 3
07: hdcp2_rx, normal, 3
08: widevinekeybox, secure, 3
09: deviceid, normal, 3
10: hdcp22_fw_private, secure, 3
11: PlayReadykeybox25, secure, 3
12: prpubkeybox, secure, 3
13: prprivkeybox, secure, 3
14: attestationkeybox, secure, 3
15: region_code, normal, 3
16: netflix_mgkid, secure, 3
17: attestationdevidbox, secure, 3
18: oemkey, normal, 3
```

初始化 unifykey

echo 1 > /sys/class/unifykeys/attach

选择操作的 unifykey

echo mac_bt > /sys/class/unifykeys/name

写入 unifykey 值

echo a4:41:1e:6a:09:bc > /sys/class/unifykeys/write

读取 unifykey 值

cat /sys/class/unifykeys/read

4.2 开机 logo 功能与客制化

4.2.1 开机 logo 图片配置

S905Y4 示例，开机 logo 文件路径：
/device/amlogic/oppen/logo_img_files/bootup.bmp。

注意 bootup.bmp 格式要求图片压缩方式为位域存储
RGB565(bmp->header.compression =03)

公版 bootup.bmp 默认格式：RGB565, 16bit；如果需要用位深度 24bit,修改如下：

bootloader/uboot-repo/bl33/v2019/board/amlogic/configs/s4_ap222.h

```
--- a/board/amlogic/configs/s4_ap222.h
+++ b/board/amlogic/configs/s4_ap222.h
@@ -79,8 +79,8 @@
         "cvbsmode=576cvbs\0" \
         "display_width=1920\0" \
         "display_height=1080\0" \
-        "display_bpp=16\0" \
-        "display_color_index=16\0" \
+        "display_bpp=24\0" \
+        "display_color_index=24\0" \
         "display_layer=osd0\0" \
         "display_color_fg=0xffff\0" \
         "display_color_bg=0\0" \
```

Logo 客制化图片生成：

1. 通过编译方式

   替换/device/amlogic/oppen/logo_img_files/bootup.bmp 后，

   make logoimg 命令编译生成

   out/target/product/ap201/upgrade/logo.img

2. 手动生成

   cd out/host/linux-x86/bin/

   ./logo_img_packer -r ../../../../device/amlogic/oppen/logo_img_files/ logo.img

   在线更换开机 logo,将板子下面路径替换 logo 图片即可

   /mnt/vendor/odm_ext/logo_files/bootup.bmp

4.2.2 Kernel logo 配置

S905Y4 示例 DTS 配置：common/arch/arm64/boot/dts/amlogic/s4_s905y4_ap222.dts

```
logo_reserved:linux,meson-fb {
    compatible = "shared-dma-pool";
    reusable;
    size = <0x0 0x800000>;
    alignment = <0x0 0x400000>;
    alloc-ranges = <0x0 0x7f800000 0x0 0x800000>;
};
```

```
&fb {
    status = "okay";
    display_size_default = <1920 1080 1920 2160 32>;
    mem_size = <0x00800000 0x1980000 0x100000>;
    logo_addr = "0x7f800000";
    mem_alloc = <0>;
    pxp_mode = <0>; /** 0:normal mode 1:pxp mode */
};
```

配置注意事项：1.logo_reserved->alloc-range 与 fb->logo_addr 分配的地址保持一致

2.fb->lmem_size 第一块大小与 logo_reserved->size 大小保持一致

3.开机 logo 图片分辨率不能大于 fb->display_size_default 分辨率配置

4.2.3 Logo 调试命令

uboot logo 显示调试命令

HDMI 初始化命令

hdmitx hpd;hdmitx get_preferred_mode;hdmitx get_parse_edid;

```
s4_ap222# hdmitx hpd;hdmitx get_preferred_mode;hdmitx get_parse_edid;
do_hpd_detect, hpd_state=1
vout_hdmi_hpd: hdmimode=1080p60hz
vout_hdmi_hpd: colorattribute=422,12bit
set outputmode: hdmimode=1080p60hz
edid_monitorcapable861: ycbcr444=1, ycbcr422=1
HDMI_EDID_BLOCK_TYPE_VENDER: prxcap->ColorDeepSupport=0x2
sink preferred_mode is 1080p60hz[16]
hdr mode is 0
dv  mode is ver:0  len: 0
hdr10+ mode is 0
edid_monitorcapable861: ycbcr444=1, ycbcr422=1
HDMI_EDID_BLOCK_TYPE_VENDER: prxcap->ColorDeepSupport=0x2
read hdmichecksum: 0x773d0000, hdmimode: 1080p60hz, colorattribute: 422,12bit
, checksum: 0x773d0000de is: 1080p60hz attr: 422,12bit
```

OSD 初始化

osd open;osd clear;

```
s4_ap222# osd open;osd clear;
[OSD]load fb addr from dts:/fb
[OSD]status disabled
[OSD]load fb addr from dts:/drm-vpu
[OSD]fb_addr for logo: 0x7f800000
[OSD]load fb addr from dts:/fb
[OSD]status disabled
[OSD]load fb addr from dts:/drm-vpu
[OSD]fb_addr for logo: 0x7f800000
not dolby stb chip 37
```

Logo 加载命令

```
s4_ap222# run load_bmp_logo
bootLogoPart=odm_ext_a
ext4logoLoadCmd=ext4load mmc 1:${logoPart} ${logoLoadAddr} ${ext4LogoPath}
259272 bytes read in 3 ms (82.4 MiB/s)
[imgread]load bmp from ext4 part okay
```

HDMI 显示 logo

vout output 1080p60hz

```
s4_ap222# vout output 1080p60hz
vpp: vpp_matrix_update: 2
set hdmitx VIC = 16
set hdmitx VIC = 16 CS = 1 CD = 6
aml_audio_init
, hdmichecksum: 0x773d000080p60hz, attr: 422,12bit
Saving Environment to STORAGE... OK
```

4.2.4 开机动画功能与客制化

4.2.5 开机动画配置

S905Y4 示例，动画文件存放路径：device/amlogic/common/products/mbox/mbox.mp4

时长	00:00:04
帧宽度	1280
帧高度	720
数据速率	2716kbps
总比特率	2833kbps
帧速率	25.00 帧/秒

动画包：device/amlogic/common/products/mbox/bootanimation.zip

存储静态图片 Android loading

动画描述配置 desc.txt

681 300 20 //动画宽度 681 动画高度 300 每秒播放 20 帧

p 1 0 android //对 android 目录静态图片循环播放 1 次，如果为 0，无限循环； 播放完动画后暂停在第 0 帧

p 0 10 loading //对 android 目录静态图片无限循环播放; 播放完动画后暂停在第 10 帧

动画显示方式配置

S905Y4 示例：device/amlogic/common/products/mbox/s4/vendor_prop.mk

```
#bootvideo
#0                      |050
#^                      |
#|                      |
#0:bootanim             |
#1:bootanim + bootvideo |
#2:bootvideo + bootanim |
#3:bootvideo            |
#others:bootanim        |
#-----------------------|050
#050:default volume value, volume range 0~100
#note that the high position 0 can not be omitted
PRODUCT_PROPERTY_OVERRIDES += \
    persist.vendor.media.bootvideo=0050
```

0050：动画 0：bootanim + :050：50%音量

4.3 ion-fb 配置

Android S 开始支持 Active Config feature。 调用标准接口切换分辨率时，FB 会重新分配。目前 surfaceflinger 是先分配新的 FB 后，再释放之前的 FB， 所以 FB 内存的个数需要配置成 6 块。

1080p

```
ion_fb_reserved:linux,ion-fb {
        compatible =  "shared-dma-pool";
        reusable;
        /* 1920x1080x4x6 round up 4M align */
        size = <0x0  0x3400000>;
        alignment = <0x0  0x400000>;
```

720P

```
ion_fb_reserved:linux,ion-fb {
        compatible =  "shared-dma-pool";
        reusable;
        /* 1280x720x4x6 round up 4M align */
        size = <0x0  0x1800000>;
        alignment = <0x0  0x400000>;
};
```

4.4 Debug 节点调试使能

调试时经常要访问/sys/kernel/debug/下的文件，Android S 版本需要在 uboot 下 Disable Selinux 后才能访问 debug 下的文件

setenv EnableSelinux permissive

saveenv

5. 安全相关

5.1 Provision key 烧录

1：生成 dgpk

```
905y4/vendor/amlogic/common/provision/tool/s4$ ./dgpk_derive.py
Generating DGPK1 and DGPK2 ...
Output:     DGPK1(dgpk1.bin) = dca543f729ccf8187f72f1af6f5def87
            DGPK2(dgpk2.bin) = 2234ecc08542ba5335635392a42b7c0c
                                                    905y4/vendor/amlogic/common/provision/tool/s4$
```

2：生成 pcpk

```
                                                    905y4/vendor/amlogic/common/provision/tool/s4$ ./pcpk_derive.py --dgpk1 dca543f729ccf8187f72f1af6f5def87
Generating PCPK ...
  Input: DGPK1 = dca543f729ccf8187f72f1af6f5def87
  Output: PCPK(s4_pcpk.bin) = b171cd3874e556cd42b30e558933bb26
```

其中 -dgpk1 参数为步骤 1 中生成的 dgpk1.bin 的 hash 值

3：生成 efuse

修改 efuse_cfg_dgpk.xml 中 DGPK_1 和 DGPK_2 的 value 值，即步骤 1 中生成的 dgpk1.bin 和 dgpk2.bin 对应的 hash 值

```
                                                    905y4/vendor/amlogic/common/provision/tool/s4$ ./gen_efuse_pattern.py --cfg efuse_cfg_dgpk.xml
Generating Efuse Pattern ...
    Input:              efue_config = efuse_cfg_dgpk.xml
    Output:       clear_efuse_pattern = s4_dgpk_efuse_pattern
           obfuscated_efuse_pattern = s4_dgpk_efuse_pattern.efuse
```

4：写入 efuse

写入 efuse 可以用放在 U 盘中写入，无 U 盘的时候可以用 adnl。

使用 U 盘：

将步骤 3 中的 s4_dgpk_efuse_pattern.efuse 放入 U 盘中，在 uboot 命令行下执行：

usb start

fatload usb 0 1080000 s4_dgpk_efuse_pattern.efuse

efuse amlogic_set 1080000

使用 adnl：

在 uboot 命令行中输入 adnl

在 windows cmd 命令行中输入

adnl partition -m mem -p 0x1080000 -f s4_dgpk_efuse_pattern.efuse

adnl oem "efuse amlogic_set 1080000"

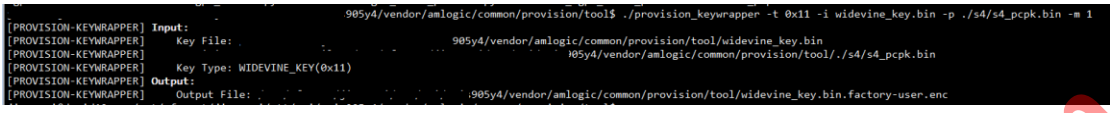
5：pcpk 加密常用 key

cd vendor/amlogic/common/provision/tool

./provision_keywrapper -l 可以查看支持的 key 列表和对应的代码

以加密 widevine_key.bin 为例

./provision_keywrapper -t 0x11 -i widevine_key.bin -p ./s4/s4_pcpk.bin -m 1



6：加密 key 烧录

将步骤 5 中生成的 widevine_key.bin.factory-user.enc 放到 U 盘中，在 kernel 的命令行中输入

tee_provision -i /storage/****/widevine_key.bin.factory-user.enc

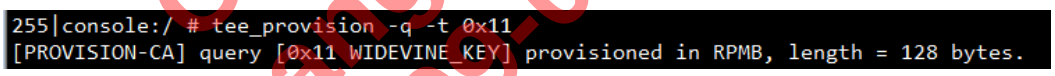
7：验证

以 widevine 和 hdcp 1.4/2.2 为例：

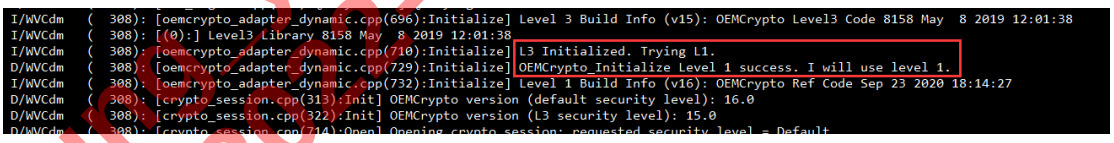
widevine key 验证：

查看是否已经烧录

tee_provision -q -t 0x11



在系统启动的时候，logcat 中会出现如下打印：

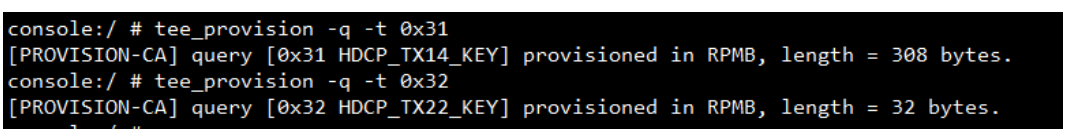


播放 ExoPlayer 中 HW secure all (L1) 视频成功。

Hdcp 1.4/2.2 key 验证：

查看是否已经烧录

tee_provision -q -t 0x31 或 0x32



拔插 hdmi 会出现如下打印

hdcp_tx begin to authenticate hdcp22:1, hdcp14:0

hdcp_tx authenticate success: 1

也可以通过如下命令查看

```
console:/ # ls -l /mnt/vendor/factory
total 20
-rwxrwxrwx 1 root root  472 2021-04-28 00:05 HDCP_TX14_KEY
-rwxrwxrwx 1 root root  196 2021-04-28 00:05 HDCP_TX22_KEY
-rwxrwxrwx 1 root root  196 2021-04-28 00:06 PLAYREADY_PRIVATE_KEY
-rwxrwxrwx 1 root root 1468 2021-04-28 00:06 PLAYREADY_PUBLIC_KEY
-rwxrwxrwx 1 root root  292 2021-04-28 00:06 WIDEVINE_KEY
```

参考文档：《Amlogic Secure Key Provision 用户手册 V2.6.pdf》

6. 常用调试

6.1 设置 uboot 环境变量

```
s4_ap222# env set env_name env_value
s4_ap222# env save
Saving Environment to STORAGE... OK
s4_ap222# env print env_name
env_name=env_value
s4_ap222#
```

设置：env set env_name env_value

保存：env save

读取：env print env_name

删除：env delete env_name

默认：env default -a

项目默认变量配置

bootloader\uboot-repo\bl33\v2019\board\amlogic\configs\s4_ap222.h

6.2 CPU 频率

查看支持的频率列表

cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies

设置最大频率

echo 1908000 > /sys/devices/system/cpu/cpu0/cpufreq/scaling_max_freq

设置最高性能模式运行

echo performance > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor

6.3 GPU 频率

查看当前频率

cat /sys/class/mpgpu/cur_freq

查看当前使用率

cat /sys/class/mpgpu/utilization

查看频率调节模式

cat /sys/class/mpgpu/scale_mode

默认 1：自动调节；2：手动调节；3：trube 模式，跑最高频

可通过 echo 修改

手动调节 gpu 频率

echo CLK_LEVEL > /sys/class/mpgpu/cur_freq CLK_LEVEL：0，1，2 .....